

UNIVERSITATEA DIN CRAIOVA

FACULTATEA DE ȘTIINȚE

SPECIALIZAREA METODE ȘI MODELE ÎN

INTELIGENȚA

ARTIFICIALĂ

LUCRARE DE DISERTAȚIE

Îndrumător științific:

Prof. univ. dr. Mirel Coșulschi

Absolvent:

MACAMETE Petre Leonard

CRAIOVA

2026

UNIVERSITATEA DIN CRAIOVA

FACULTATEA DE ȘTIINȚE

SPECIALIZAREA METODE ȘI MODELE ÎN

INTELIGENȚA

ARTIFICIALĂ

Sistem de eLearning bazat pe blockchain

Îndrumător științific:

Prof. univ. dr. Mirel Coșulschi

Absolvent:

MACAMETE Petre Leonard

CRAIOVA

2026

Cuprins

1	Introducere	6
1.1	Contextul lucrării	6
1.2	Motivația cercetării	7
1.3	Problema abordată	8
1.4	Obiective	8
1.5	Contribuții	9
1.6	Structura lucrării	10
	Partea I	11
2	Stadiul cunoașterii în eLearning și Web3	12
2.1	Evoluția aplicațiilor de eLearning	12
2.2	Principii de proiectare pentru experiența educațională	12
2.3	Gamificare și motivație	13
2.4	Modelul learn-to-earn	14
2.5	Aplicații hibride Web2/Web3	14
2.6	Tehnologii imersive și învățare socială	16
2.7	Repere pedagogice pentru învățarea autonomă	16
2.8	Feedback, vizibilitate și reducerea efortului cognitiv	17
2.9	Tipologia recompenselor educaționale	18
2.10	Riscuri etice ale modelului learn-to-earn	19
2.11	Criterii pentru folosirea blockchain-ului	20
2.12	Antifraudă în platformele educaționale recompensate	20
2.13	Poziționarea RustLearn față de soluțiile existente	20
2.14	Indicatori pentru evaluarea unei platforme recompensate	21
2.15	Concluzii ale analizei	22
3	Fundamente tehnologice și justificarea alegerilor	23

3.1	Criterii de selecție	23
3.2	Rust pentru backend	23
3.3	Actix Web pentru API.....	24
3.4	PostgreSQL și Diesel	24
3.5	Stocare S3 și worker pentru conținut multimedia	25
3.6	Next.js pentru frontend	25
3.7	Ethereum, Solidity și OpenZeppelin	26
3.8	Arhitectură modulară monolitică.....	26
3.9	Alternative analizate pentru backend	27
3.10	Alternative analizate pentru frontend.....	27
3.11	Contractul dintre straturi	28
3.12	Date off-chain și evenimente on-chain	29
3.13	Costuri operaționale și costuri cognitive.....	29
3.14	De ce monolit modular, nu microservicii premature.....	30
3.15	Mentenabilitate și evoluție	30
3.16	Securitatea prin alegerea tehnologiilor	31
3.17	Portabilitate și independență operațională	31
3.18	Criterii de respingere a alternativelor.....	32
3.19	Comparație sintetică	32
3.20	Concluzie	33
	Partea a II-a	34
4	Analiza și arhitectura sistemului RustLearn	35
4.1	Descriere generală	35
4.2	Actorii sistemului	35
4.3	Cerințe funcționale.....	36
4.4	Cerințe nefuncționale	36
4.5	Arhitectura de ansamblu	37
4.6	Straturile aplicației.....	37
4.7	Modelul de permisiuni	38
4.8	Fluxul de recompensare	39
4.9	Persistența datelor	40
4.10	Use case-uri critice	40

4.11	Fluxul de finalizare a unui curs recompensat	41
4.12	Modelarea stărilor și invariantelor	42
4.13	Frontiere între domenii	43
4.14	Arhitectura rapoartelor și exporturilor	43
4.15	Valoarea pentru fiecare actor	43
4.16	Scalabilitate conceptuală	44
4.17	Analiza cazurilor de eroare	44
4.18	Decizii privind granularitatea auditului	45
4.19	Valoarea arhitecturii	45
4.20	Concluzie	46
5	Implementarea platformei	47
5.1	Pornirea aplicației și compunerea stării	47
5.2	API-ul și middleware-ul	48
5.3	Identitate, parole și verificare email	48
5.4	Modelul cursurilor și al progresului	48
5.5	Organizații și aplicații de profesor	49
5.6	Recompense și politici	49
5.7	Portofele și tranzacții	50
5.8	Contractele inteligente	50
5.9	Worker, procesare video și indexare	51
5.10	Frontend-ul produsului	51
5.11	Notificări și feedback operațional	51
5.12	Implementarea use case-urilor de aplicație	52
5.13	Idempotentă și consistență	52
5.14	Implementarea permisiunilor pe scopuri	53
5.15	Implementarea recompenselor ca proces	53
5.16	Integrarea contractelor inteligente	54
5.17	Implementarea portofelelor	55
5.18	Worker-ul și toleranța la eșec	55
5.19	Implementarea frontend-ului pe stări	55
5.20	KYC, audit și operații administrative	56
5.21	Observabilitate și administrare locală	56

5.22	Fluxuri concrete de API	57
5.23	Validarea datelor la intrare	57
5.24	Implementarea exporturilor și rapoartelor	57
5.25	Corelarea frontend-backend	58
5.26	Configurare și separarea mediilor	58
5.27	Calitatea codului și verificări înainte de livrare	59
5.28	Concluzie	59
6	Validare, operare și limitări	60
6.1	Strategia de validare	60
6.2	Teste backend	60
6.3	Migrații și schema bazei de date	61
6.4	Health și readiness	61
6.5	Docker Compose și mediu local	62
6.6	Securitate și control al riscurilor	62
6.7	Limitări ale implementării	63
6.8	Scenarii de testare recomandate	63
6.9	Criterii de acceptanță	64
6.10	Matrice de riscuri	65
6.11	Testare manuală end-to-end	65
6.12	Plan de deployment	66
6.13	Backup, restaurare și migrații	66
6.14	Performanță și scalare	67
6.15	Conformitate și protecția datelor	67
6.16	Limitări de cercetare și validare empirică	67
6.17	Indicatori recomandați pentru monitorizare	68
6.18	Pregătirea susținerii și demonstrației	68
6.19	Criterii de maturitate pentru continuarea proiectului	69
6.20	Concluzie	69
7	Concluzii și direcții viitoare	70
7.1	Sinteza lucrării	70
7.2	Contribuții realizate	70
7.3	Răspuns la problema inițială	71

7.4	Limitări	71
7.5	Directii viitoare	72
7.6	Concluzie finală	72
Bibliografie		73

Capitolul 1

Introducere

1.1 Contextul lucrării

Digitalizarea educației a schimbat modul în care sunt proiectate cursurile, modul în care este urmărit progresul și modul în care sunt evaluate competențele. Platformele de eLearning au făcut posibilă distribuirea rapidă a materialelor, organizarea cursurilor în ritm asincron și extinderea accesului la comunități care nu pot participa constant la activități față în față. Totuși, o limitare importantă rămâne felul în care progresul educațional este perceput de cursant. În multe sisteme, rezultatele apar sub forma unor note, bare de progres sau certificate, dar recompensa este întârziată, dificil de verificat independent și rareori integrată într-un mecanism economic transparent.

Tema lucrării, *Sistem de eLearning bazat pe blockchain*, pleacă de la ideea că activitatea educațională poate fi însoțită de un mecanism de recompensare verificabil, fără ca procesul didactic să fie redus la o simplă competiție pentru puncte. Tehnologia blockchain permite înregistrarea unor transferuri și evenimente într-un mod transparent, iar contractele inteligente pot automatiza reguli de emitere, transfer sau verificare a unor jetoane digitale [19, 30]. În același timp, o platformă educațională are nevoie de operații rapide, control al accesului, stocare eficientă a conținutului multimedia, fluxuri administrative și protecția datelor personale. Din acest motiv, soluția analizată în lucrare nu mută toate operațiile în blockchain, ci combină un nucleu web clasic cu un strat de recompense blockchain.

Modelul *learn-to-earn* a devenit vizibil prin platforme care recompensează utilizatorii pentru parcurgerea unor resurse educaționale sau pentru participarea la activități de învățare. Exemple precum BitDegree [17] sau Phemex Learn Crypto [28] arată interesul pentru legătura dintre învățare și recompense digitale, însă aceste soluții sunt frecvent concentrate în jurul conținutului despre criptomonede. Lucrarea propune extinderea acestei logici către o platformă generală

de eLearning, în care cursurile, evaluările, organizațiile, rolurile și recompensele pot funcționa împreună într-un sistem coerent.

1.2 Motivația cercetării

Motivația principală este aceea de a proiecta o platformă în care progresul educațional să fie măsurabil, verificabil și util pentru mai mulți actori: cursanți, profesori, organizații și administratori. Pentru cursant, recompensa poate funcționa ca o confirmare imediată a efortului depus. Pentru profesor, sistemul oferă un mod de a defini activități eligibile pentru recompense și de a valida performanța. Pentru organizații, platforma poate susține programe educaționale interne, bugete de recompensare și rapoarte privind participarea. Pentru administratori, arhitectura trebuie să ofere audit, control al fraudelor, reconciliere și trasabilitate.

Importanța socială a accesului la educație este legată de reducerea unor cercuri de vulnerabilitate. Lipsa educației reduce șansele de angajare, veniturile și participarea socială, iar aceste efecte pot întreține sărăcia și excluziunea. Figura 1.1 ilustrează relația dintre sărăcie, sănătate, educație și capital uman. În acest context, o platformă educațională accesibilă și stimulativă nu rezolvă singură aceste probleme, dar poate contribui la reducerea barierelor prin acces la cursuri, feedback și recunoașterea progresului.

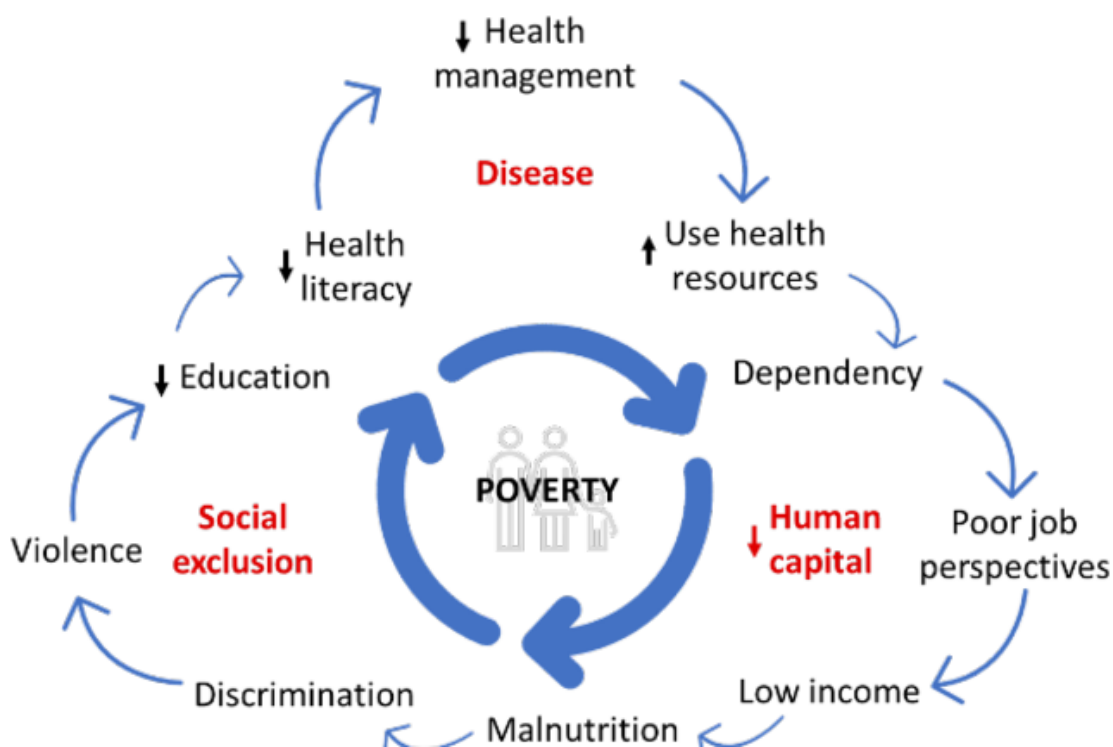


Figura 1.1. Impacturi asociate cercului vicios al sărăciei.¹

Recompensele cu jetoane digitale trebuie tratate cu prudență. Dacă sunt implementate superficial, ele pot încuraja comportamente oportuniste, colectarea artificială de puncte sau dependența de motivație extrinsecă. De aceea, lucrarea pune accent pe un flux de recompensare auditat: un eveniment educațional produce un candidat la recompensă, eligibilitatea este verificată, suma este aprobată, tranzacția este planificată, iar creditarea portofelului intern se face numai după confirmarea etapei relevante. În acest mod, recompensa devine o consecință a progresului educațional durabil, nu un substitut al învățării.

1.3 Problema abordată

Problema analizată constă în proiectarea și implementarea unui sistem de eLearning care să integreze recompense blockchain fără a compromite performanța, securitatea și controlul administrativ necesare unei platforme educaționale. Un sistem exclusiv descentralizat ar oferi transparență, dar ar fi costisitor, lent și dificil de adaptat pentru operații educaționale frecvente. Un sistem exclusiv centralizat ar fi mai simplu de operat, dar ar pierde valoarea auditabilă a transferurilor tokenizate și ar depinde complet de încrederea în operatorul platformei.

Soluția propusă este o arhitectură hibridă. Backend-ul Rust/Actix Web gestionează autentificarea, permisiunile, cursurile, progresul, evaluările, organizațiile și operațiile administrative. PostgreSQL păstrează starea operațională și istoricul necesar raportării. Stocarea compatibilă S3 gestionează fișierele de curs, iar un worker separat procesează materialele video. Contractele Solidity și integrarea Ethereum gestionează LearnToken, importul de tokenuri și transferurile semnate. Frontend-ul Next.js oferă interfețe diferențiate pentru cursanți, profesori, organizații și administratori.

1.4 Obiective

Obiectivul general al lucrării este realizarea unei platforme de eLearning bazate pe blockchain, capabilă să urmărească activitatea educațională și să susțină recompense tokenizate într-un mod controlat. Obiectivele specifice sunt:

- analiza stadiului actual privind eLearning-ul, gamificarea și aplicațiile hibride Web2/Web3;

¹Sursa: Bell, Victoria et al., *Community Health of Children and Adolescents in Sub-Saharan Africa*, European Journal of Medical and Health Sciences, 2023 [1].

- definirea unei arhitecturi care separă domeniul educațional, logica aplicației, infrastructura și interfața HTTP;
- justificarea alegerilor tehnologice pentru backend, frontend, bază de date, stocare, worker și contracte inteligente;
- implementarea unui model de roluri și permisiuni pe trei niveluri: platformă, organizație și curs;
- proiectarea unui flux de recompensare cu stări auditate, reconciliere și protecție împotriva dublării recompenselor;
- integrarea unui token ERC-20 cu suport pentru operații semnate și import de fonduri prin standarde Ethereum;
- validarea soluției prin teste, migrații reversibile, verificări de readiness și scenarii de operare locală.

1.5 Contribuții

Contribuțiile lucrării sunt atât conceptuale, cât și aplicative. Din punct de vedere conceptual, lucrarea arată de ce o arhitectură hibridă este mai potrivită pentru eLearning decât o descentralizare completă. Datele educaționale, permisiunile și procesele administrative rămân într-o bază relațională controlabilă, iar blockchain-ul este folosit pentru valoarea sa specifică: transferuri verificabile, audit public al tokenului și interoperabilitate cu portofele externe. Din punct de vedere aplicativ, lucrarea documentează proiectarea și implementarea platformei RustLearn, cu accent pe separarea responsabilităților, trasabilitate și extensibilitate.

O contribuție importantă este modelarea recompenselor ca proces, nu ca simplă tranzacție. Între finalizarea unei activități de învățare și transferul efectiv al tokenurilor apar verificări, aprobări, stări intermediare și operații de reconciliere. Această abordare reduce riscul de fraudă, permite intervenții administrative și face posibilă explicarea fiecărei decizii. De asemenea, lucrarea evidențiază rolul worker-ului în mutarea operațiilor costisitoare în afara cererilor HTTP, precum procesarea video și indexarea depunerilor de tokenuri.

1.6 Structura lucrării

Lucrarea este organizată conform ghidului de elaborare pentru disertații. Capitolul 1 prezintă tema, motivația, problema abordată, obiectivele și contribuțiile. Partea I analizează stadiul cunoașterii: Capitolul 2 discută eLearning-ul, gamificarea, modelul learn-to-earn, riscurile anti-fraudă și aplicațiile hibride Web2/Web3, iar Capitolul 3 justifică tehnologiile utilizate și compară alternativele relevante. Partea a II-a prezintă rezultatele aplicative: Capitolul 4 descrie analiza și arhitectura sistemului RustLearn, Capitolul 5 detaliază implementarea backend-ului, frontend-ului, worker-ului și contractelor inteligente, iar Capitolul 6 descrie testarea, operarea, securitatea și limitările. Capitolul 7 sintetizează concluziile și direcțiile de dezvoltare viitoare.

Partea I

Capitolul 2

Stadiul cunoașterii în eLearning și Web3

2.1 Evoluția aplicațiilor de eLearning

Aplicațiile de eLearning au evoluat de la simple depozite de fișiere și teste online către sisteme complexe care gestionează parcursuri educaționale, progres, evaluări, comunicare, notificări și rapoarte. Într-o platformă modernă, experiența de învățare nu este definită doar de conținut, ci și de felul în care utilizatorul primește feedback, își vede progresul, revine la materialele relevante și interacționează cu profesorii sau cu organizația care oferă cursul. De aceea, proiectarea aplicațiilor de eLearning trebuie analizată simultan din perspectivă pedagogică, tehnică și operațională.

Un sistem educațional digital trebuie să rezolve mai multe nevoi. Cursantul are nevoie de acces rapid la lecții, de navigare clară, de feedback și de un mod de a înțelege ce trebuie făcut în continuare. Profesorul are nevoie de instrumente de creare și publicare a conținutului, de evaluare, de observare a progresului și de moderare. Organizațiile au nevoie de administrarea membrilor, cursurilor și rapoartelor. Administratorii platformei au nevoie de control asupra rolurilor, auditului, securității și integrărilor. Aceste cerințe duc la o concluzie importantă: eLearning-ul nu poate fi tratat ca o simplă interfață web, ci ca un sistem socio-tehnic cu fluxuri de lucru diferite.

2.2 Principii de proiectare pentru experiența educațională

O interfață de eLearning trebuie să fie ușor de înțeles, previzibilă și accesibilă. Principiile UI/UX discutate în literatura de specialitate pun accent pe claritatea navigării, reducerea efortului cognitiv și coerența componentelor vizuale [25, 27]. Într-un context educațional, aceste principii au o miză suplimentară: dacă interfața este confuză, cursantul consumă energie pentru orientare, nu pentru învățare. Astfel, ecranul principal trebuie să indice cursurile active, lecțiile următoare, statusul

evaluărilor și eventualele notificări importante.

Personalizarea este un alt principiu relevant. Platforma trebuie să afișeze informații diferite pentru un cursant, un profesor, un administrator de organizație sau un operator de platformă. Personalizarea nu înseamnă doar preferințe vizuale, ci adaptarea suprafeței de lucru la permisiunile reale ale utilizatorului. Dacă un profesor are acces la un curs, interfața trebuie să afișeze conținutul, înscrierile și coada de recompense pentru acel curs. Dacă utilizatorul are doar rol de cursant, aceeași platformă trebuie să ascundă operațiunile administrative și să evidențieze lecțiile, progresul și istoricul recompenselor.

Un design eficient pentru eLearning trebuie să mențină echilibrul dintre densitatea informației și claritatea acțiunilor. Platformele educaționale sunt folosite repetat, iar utilizatorii au nevoie de interfețe stabile, cu filtre, stări goale explicite, mesaje de eroare inteligibile și confirmări pentru operații cu efect permanent. Aceste principii sunt relevante pentru RustLearn, deoarece sistemul include rute separate pentru cursanți, profesori, organizații și administratori, fiecare cu propriile decizii și responsabilități.

2.3 Gamificare și motivație

Gamificarea reprezintă utilizarea elementelor inspirate din jocuri în contexte care nu sunt jocuri. Deterding et al. definesc gamificarea ca folosire a elementelor de design de joc în contexte non-ludice [5]. În eLearning, aceste elemente pot include puncte, niveluri, badge-uri, streak-uri, clasamente, provocări și feedback imediat. Scopul nu este transformarea cursului într-un joc, ci crearea unor repere motivaționale care să ajute cursantul să continue, să observe progresul și să primească recunoaștere.

Mecanismele de recompensare trebuie proiectate atent. Un sistem bazat exclusiv pe puncte poate produce comportamente superficiale, în care utilizatorul optimizează pentru recompensă, nu pentru înțelegere. Cercetările privind învățarea online arată importanța angajamentului, a participării și a feedback-ului în menținerea activității educaționale [12, 10, 7]. De aceea, recompensele trebuie legate de evenimente educaționale durabile: finalizarea unei evaluări, promovarea unui test, parcurgerea verificabilă a unui curs sau aprobarea manuală a unei activități relevante.

Figura 2.1 prezintă relația dintre activitatea de învățare și recompensa tokenizată. Elementul central este verificarea. Activitatea produce dovezi, dovezile sunt evaluate prin reguli și prin decizii autorizate, iar recompensa este acordată numai dacă rezultatul este valid. Astfel, gamificarea nu înlocuiește evaluarea, ci o face mai vizibilă și mai motivantă.

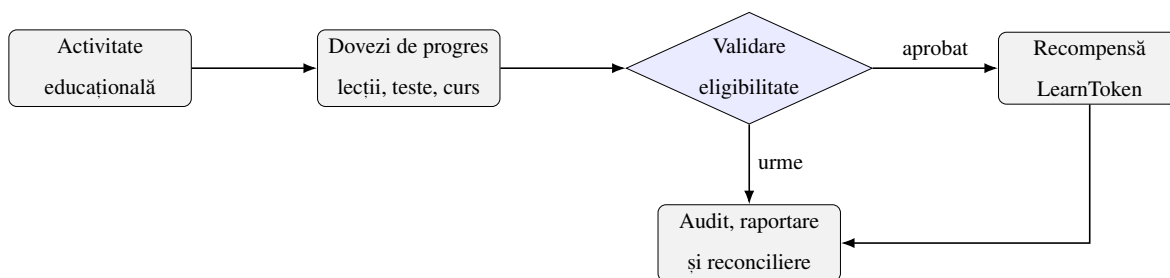


Figura 2.1. Legătura dintre învățare, validare și recompensă tokenizată (elaborare proprie).

2.4 Modelul learn-to-earn

Modelul *learn-to-earn* propune recompensarea cursanților pentru activitate educațională verificată. El este apropiat de gamificare, dar introduce o componentă economică explicită. În loc ca recompensa să fie doar un badge intern, aceasta poate lua forma unui jeton digital care are un istoric verificabil și care poate fi legat de portofele, bugete, reguli de transfer sau politici de retragere. Exemplele existente, precum BitDegree și Phemex Learn Crypto, arată că utilizatorii pot fi atrași de cursuri atunci când parcurgerea conținutului are o recompensă clară [17, 28].

Limitarea multor platforme learn-to-earn este concentrarea pe conținut legat de domeniul cripto. Această concentrare este firească în fazele inițiale ale ecosistemului, deoarece utilizatorii familiarizați cu portofele și tokenuri sunt mai dispuși să accepte astfel de mecanisme. Totuși, ideea poate fi extinsă la educația STEM, la programe de recalificare, la cursuri organizaționale și la programe de învățare continuă. În aceste cazuri, recompensa nu trebuie văzută ca speculație financiară, ci ca instrument de recunoaștere și stimulare.

Un model matur learn-to-earn trebuie să includă controale împotriva abuzului. Dacă un cursant poate obține recompense repetate pentru același eveniment sau poate produce dovezi artificiale de progres, sistemul își pierde credibilitatea. De aceea, sunt necesare chei de idempotență, politici de recompensare versionate, stări intermediare și mecanisme de blocare pentru profesori, organizații, cursuri sau politici suspecte. Aceste cerințe depășesc simpla gamificare și transformă recompensa într-un proces administrativ și tehnic.

2.5 Aplicații hibride Web2/Web3

O aplicație hibridă combină infrastructura centralizată a aplicațiilor web cu mecanisme descentralizate precum blockchain-ul, contractele inteligente și portofelele cripto. Literatura despre Web3 subliniază ideea de control asupra datelor, proprietate digitală și aplicații descentralizate [9, 24].

În practică, însă, multe aplicații au nevoie de componente Web2 pentru performanță, experiență de utilizare și conformitate operațională. Această combinație este valoroasă în special pentru eLearning, unde majoritatea operațiilor sunt frecvente, sensibile la latență și dependente de permisiuni.

Brave Browser este un exemplu de aplicație hibridă: oferă o experiență web familiară, dar integrează portofel, mecanisme de confidențialitate și model de recompensare prin tokenuri [18, 20]. Valoarea exemplului nu constă în identitatea exactă a produsului, ci în principiul arhitectural: utilizatorul nu trebuie forțat să înțeleagă toate detaliile blockchain pentru a beneficia de funcțiile Web3. În același mod, un cursant RustLearn trebuie să poată urmări lecții și recompense fără ca fiecare acțiune educațională să devină o tranzacție blockchain.

Tabelul 2.1 sintetizează diferențele dintre cele trei variante arhitecturale relevante pentru lucrare.

Tabela 2.1. Comparatie între abordări centralizate, descentralizate și hibride (elaborare proprie).

Criteriu	Sistem centralizat	Sistem descentralizat	Sistem hibrid
Performanță	Foarte bună pentru operații frecvente și interogări complexe	Depinde de rețea, costuri și confirmări	Bună pentru fluxuri educaționale, cu blockchain folosit selectiv
Audit public	Limitat la logurile operatorului	Puternic pentru evenimente on-chain	Puternic pentru tokenuri, completat de audit intern
Cost operațional	Predictibil, mai ales pentru date și fișiere	Costuri pe tranzacții și contracte	Costuri controlate prin alegerea evenimentelor on-chain
Experiență utilizator	Familiară și rapidă	Poate necesita portofel și semnături dese	Familiară, cu opțiuni Web3 când sunt necesare
Control administrativ	Ridicat	Limitat de regulile contractului	Ridicat pentru educație, transparent pentru recompense

2.6 Tehnologii imersive și învățare socială

Pe lângă gamificare și recompense, eLearning-ul modern folosește simulări, realitate augmentată, realitate virtuală și comunități de învățare. Studiile despre realitatea virtuală în educația superioară arată potențialul experiențelor imersive pentru domenii în care practica este greu de reprodus în siguranță sau la scară mare [13, 6]. Simulările și jocurile educaționale pot transforma concepte abstracte în situații aplicabile [15, 4].

În cazul RustLearn, lucrarea nu implementează AR/VR, dar arhitectura ia în calcul conținut multimedia, procesare video și stocare de obiecte. Această alegere este importantă deoarece o platformă de eLearning nu trebuie să fie limitată la text. Lecțiile video, fișierele atașate, resursele audio și materialele interactive pot fi integrate progresiv dacă infrastructura suportă încărcare, procesare și livrare eficientă.

Învățarea socială completează componenta individuală. Forumurile, comentariile, discuțiile moderate și colaborarea între cursanți pot crește angajamentul și pot oferi contexte de aplicare a cunoștințelor [7, 10]. Într-un sistem cu recompense, aceste funcții trebuie moderate cu grijă, deoarece activitatea socială poate deveni sursă de abuz dacă este recompensată fără verificare. De aceea, RustLearn tratează permisiunile, moderarea și auditul ca elemente de bază, nu ca funcții secundare.

2.7 Repere pedagogice pentru învățarea autonomă

Un sistem de eLearning eficient trebuie să susțină învățarea autonomă, dar autonomia nu înseamnă abandonarea cursantului în fața unei liste de materiale. Autonomia presupune ghidaj, vizibilitate asupra progresului și posibilitatea de a reveni asupra conținutului atunci când rezultatul unei evaluări indică lacune. Cercetările privind învățarea online arată că participarea, feedback-ul și percepția utilității activităților influențează implicarea cursantului [7, 10]. Prin urmare, platforma trebuie să facă explicit traseul: ce lecție urmează, ce condiții trebuie îndeplinite, ce evaluare validează competența și ce recompensă poate fi obținută în mod legitim.

În cazul unei platforme cu recompense, proiectarea pedagogică are o responsabilitate suplimentară. Dacă recompensele sunt vizibile, ele pot orienta atenția cursantului către rezultatul extrinsec. Pentru a evita acest efect, recompensa trebuie plasată după verificarea învățării, nu înaintea ei. Cursantul trebuie să înțeleagă că tokenul este o consecință a progresului și nu scopul unic al activității. Din acest motiv, RustLearn leagă recompensele de evenimente care pot fi moti-

vate educațional: finalizarea unui curs, promovarea unei evaluări, aprobarea unei activități de către profesor sau participarea la un program organizațional cu reguli clare.

Un alt reper pedagogic este autoreglarea. Paans et al. discută variații temporale în învățarea autoreglată în medii hipertextuale [12], iar această observație este relevantă pentru platformele cu trasee neliniare. Cursantul poate intra într-un curs din catalog, poate reveni la un capitol, poate repeta un test sau poate consulta istoricul recompenselor. Sistemul trebuie să păstreze coerența acestor acțiuni. Dacă progresul este fragmentat sau greu de interpretat, utilizatorul pierde sentimentul de control. Dacă, dimpotrivă, platforma explică fiecare stare, autonomia devine practicabilă.

2.8 Feedback, vizibilitate și reducerea efortului cognitiv

Designul interfeței are efect direct asupra calității învățării. O interfață încărcată obligă utilizatorul să consume energie mentală pentru orientare, iar o interfață prea săracă ascunde informații importante. Ghidurile de utilizabilitate recomandă coerență, vizibilitatea stării sistemului și feedback clar pentru acțiunile utilizatorului [14, 25]. Într-o platformă educațională, aceste principii trebuie aplicate la nivel de traseu: cursantul trebuie să vadă ce cursuri are active, unde s-a oprit, ce evaluări sunt disponibile și ce recompense sunt în așteptare.

Feedback-ul trebuie diferențiat după tipul de acțiune. Pentru o acțiune reversibilă, un mesaj scurt este suficient. Pentru o operație cu efect economic, precum aprobarea unei recompense, mesajul trebuie să indice starea exactă și următorul pas. De exemplu, un candidat la recompensă poate fi *submitted*, *teacher approved*, *amount approved*, *token confirmed* sau *wallet credited*. Dacă interfața traduce aceste stări în mesaje clare, utilizatorul înțelege că întârzierea nu înseamnă pierderea recompensei, ci parcurgerea unui flux controlat.

Figura 2.2 sintetizează legătura dintre acțiunea cursantului, feedback și autoreglare. Observația centrală este că feedback-ul nu este doar o reacție vizuală, ci un mecanism de orientare. El spune utilizatorului dacă activitatea a fost înregistrată, dacă este suficientă pentru progres, dacă trebuie repetată sau dacă urmează validarea profesorului.

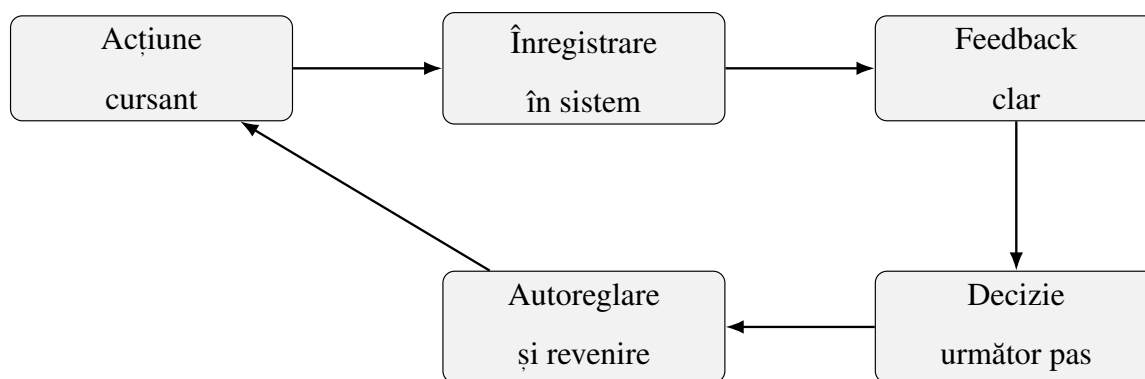


Figura 2.2. Feedback-ul ca mecanism de autoreglare în eLearning (elaborare proprie).

2.9 Tipologia recompenselor educaționale

Recompensele din eLearning pot fi împărțite în mai multe categorii. Prima categorie este simbolică: badge-uri, titluri, niveluri și certificate. Acestea oferă recunoaștere, dar rămân de regulă în interiorul platformei. A doua categorie este funcțională: deblocarea unor lecții, accesul la materiale avansate, prioritate la mentorat sau dreptul de a participa la proiecte. A treia categorie este economică: puncte convertibile, cupoane, tokenuri sau beneficii cu valoare externă. RustLearn se află la intersecția dintre recunoașterea educațională și recompensa economică, deoarece LearnToken este gândit ca semn verificabil al unui progres validat.

Această tipologie este importantă deoarece fiecare categorie are riscuri diferite. Recompensele simbolice sunt ușor de acordat, dar pot deveni lipsite de valoare dacă sunt prea frecvente. Recompensele funcționale pot susține trasee educaționale, dar trebuie proiectate astfel încât să nu blocheze cursanții care au nevoie de recuperare. Recompensele economice au cel mai mare potențial motivațional, dar și cel mai mare risc de abuz. Ele cer politici clare, limite, audit și capacitatea de a opri temporar fluxurile suspecte.

Tabela 2.2. Tipuri de recompense și implicații pentru platformă (elaborare proprie).

Tip recompensă	Avantaj	Risc controlat prin arhitectură
Simbolică	Recunoaștere rapidă, ușor de înțeles de cursant	Inflație de badge-uri și pierdere relevanței
Funcțională	Încurajează trasee progresive și implicare continuă	Blocarea artificială a accesului la învățare
Economică	Creează stimulent verificabil și portabil	Fraudă, dublare, dispute și costuri de tranzacție
Socială	Sustine colaborarea și reputația în comunitate	Manipulare prin interacțiuni artificiale

2.10 Riscuri etice ale modelului learn-to-earn

Modelul learn-to-earn trebuie evaluat și etic, nu doar tehnic. O primă problemă este echitatea. Dacă recompensele depind de timp disponibil, dispozitive performante sau acces constant la internet, platforma poate amplifica diferențe existente. O a doua problemă este motivația. Recompensa extrinsecă poate susține implicarea pe termen scurt, dar poate slăbi interesul pentru conținut dacă este percepută ca singurul motiv de participare. O a treia problemă este transparența: utilizatorul trebuie să știe ce date sunt colectate, ce evenimente contează și de ce primește sau nu primește tokenuri.

Pentru RustLearn, aceste riscuri conduc la decizii arhitecturale concrete. Regulile de recompensare trebuie să fie explicabile, politicile trebuie să fie versionate, iar istoricul deciziilor trebuie să poată fi consultat. În plus, sistemul trebuie să permită intervenție umană. O platformă complet automată ar putea acorda recompense rapid, dar ar fi mai vulnerabilă la situații neprevăzute. Într-o platformă educațională, profesorul și administratorul rămân actori importanți tocmai pentru că pot interpreta contextul.

Un alt aspect etic este separarea dintre evaluarea educațională și valoarea financiară. Un cursant nu trebuie să fie penalizat economic pentru o eroare tehnică a platformei, iar o recompensă nu trebuie să fie folosită pentru a ascunde calitatea slabă a conținutului. De aceea, lucrarea tratează recompensa ca un strat suplimentar peste eLearning, nu ca înlocuitor al pedagogiei. Conținutul, evaluarea, feedback-ul și suportul profesorului rămân elementele centrale ale experienței educaționale.

2.11 Criterii pentru folosirea blockchain-ului

Blockchain-ul este justificat numai atunci când avantajele sale depășesc costurile. Pentru date care se modifică frecvent, care conțin informații personale sau care trebuie șterse la cerere, o bază de date relațională este mai potrivită. Pentru transferuri de tokenuri, evenimente auditate public și interoperabilitate cu portofele, blockchain-ul poate aduce valoare reală [19, 9]. Această distincție este esențială pentru o arhitectură matură.

În RustLearn, criteriul principal este finalitatea evenimentului. O lecție vizualizată nu trebuie scrisă pe blockchain, deoarece este un eveniment frecvent și sensibil contextual. O recompensă aprobată și confirmată poate avea reprezentare on-chain, deoarece devine parte din istoricul economic al utilizatorului. În același timp, decizia care a condus la recompensă rămâne explicată în baza de date, prin audit intern. Rezultatul este un model dublu: blockchain-ul confirmă transferul, iar aplicația explică motivul transferului.

2.12 Antifraudă în platformele educaționale recompensate

Frauda într-o platformă de eLearning recompensată poate apărea în mai multe forme. Un utilizator poate încerca să finalizeze același eveniment de mai multe ori, să automatizeze parcurgerea lecțiilor, să exploateze o evaluare prea simplă sau să colaboreze cu un profesor pentru aprobări nejustificate. O organizație poate configura politici prea generoase pentru a muta tokenuri fără activitate reală. Un atacator tehnic poate încerca să reutilizeze semnături, să modifice adrese de portofel sau să forțeze stări inconsistente.

Aceste riscuri nu pot fi eliminate printr-o singură tehnologie. Ele cer o combinație de reguli de domeniu, constrângeri de bază de date, audit, permisiuni, monitorizare și controale blockchain. Cheile de idempotentă previn repetarea aceleiași operații, politicile active definesc limitele acordării, fraud blocks pot opri actori sau cursuri suspecte, iar nonce-urile din contracte previn replay-ul semnăturilor. În acest sens, securitatea nu este un modul separat, ci o proprietate transversală a arhitecturii.

2.13 Poziționarea RustLearn față de soluțiile existente

RustLearn se diferențiază de platformele de eLearning clasice prin introducerea unui strat verificabil de recompense, dar se diferențiază și de aplicațiile Web3 pure prin păstrarea logicii educaționale

într-un sistem controlat. Platformele clasice pot gestiona cursuri, progres și evaluări, dar recompensele rămân de regulă interne. Aplicațiile Web3 pot gestiona tokenuri și proprietate digitală, dar nu oferă automat instrumente pedagogice mature. RustLearn încearcă să combine cele două direcții fără a confunda rolurile lor.

Această poziționare este importantă pentru tema lucrării deoarece arată că blockchain-ul nu este folosit decorativ. Valoarea lui apare în punctele în care un token trebuie emis, transferat, ars, importat sau asociat unui portofel. În rest, platforma folosește mecanisme consacrate: autentificare, permisiuni, PostgreSQL, stocare de obiecte și worker asincron. Rezultatul este o soluție care poate fi înțeleasă atât de un evaluator tehnic, cât și de un evaluator preocupat de utilitatea educațională.

2.14 Indicatori pentru evaluarea unei platforme recompensate

Stadiul cunoașterii arată că o platformă educațională recompensată nu trebuie evaluată doar prin numărul de utilizatori sau prin volumul tokenurilor distribuite. Indicatorii relevanți trebuie să măsoare progresul educațional, calitatea implicării și robustețea mecanismului de recompensare. Rata de finalizare a cursurilor, numărul de reveniri, scorurile la evaluări, timpul mediu până la feedback și satisfacția cursanților sunt indicatori pedagogici. Numărul de candidați respinși, recompensele reconciliate, fraud blocks active și timpul până la creditarea wallet-ului sunt indicatori operaționali.

Această separare între indicatori pedagogici și indicatori operaționali previne o interpretare greșită a succesului. O platformă care distribuie multe tokenuri poate avea totuși învățare superficială. O platformă cu multe respingeri poate avea fie controale eficiente, fie reguli prost comunicate. De aceea, RustLearn trebuie analizată prin corelarea indicatorilor. Dacă finalizările cresc, dar scorurile scad, recompensa poate încuraja parcurgerea rapidă. Dacă timpul de creditare crește, problema poate fi în worker, în blockchain sau în fluxul de aprobare. Indicatorii devin astfel instrumente de învățare organizațională, nu doar statistici.

Tabela 2.3. Indicatori recomandați pentru evaluarea modelului learn-to-earn (elaborare proprie).

Categorie	Exemple de indicatori	Interpretare
Pedagogică	finalizări, scoruri, reveniri, timp pe lecții	Arată dacă recompensa susține învățarea reală
Operațională	timp de aprobare, timp de creditare, joburi eșuate	Arată dacă sistemul funcționează previzibil
Antifraudă	candidați respinși, blocaje, reconciliere	Arată calitatea controalelor și a regulilor
Experiență utilizator	erori afișate, stări goale, abandon în flux	Arată dacă interfața explică suficient procesul

2.15 Concluzii ale analizei

Analiza stadiului cunoașterii conduce la trei observații relevante pentru proiectarea platformei. În primul rând, eLearning-ul eficient are nevoie de interfețe clare, feedback și trasee adaptate rolului utilizatorului. În al doilea rând, gamificarea și recompensele pot crește motivația, dar trebuie legate de activități educaționale reale și de verificări antifraudă. În al treilea rând, blockchain-ul aduce valoare atunci când este folosit pentru ceea ce face bine: tokenuri, trasabilitate și interoperabilitate. Pentru cursuri, roluri, fișiere, rapoarte și administrare, o infrastructură web centralizată rămâne mai potrivită. Aceste observații fundamentează arhitectura hibridă prezentată în capitolele următoare.

Capitolul 3

Fundamente tehnologice și justificarea alegerilor

3.1 Criterii de selecție

Alegerea tehnologiilor pentru un sistem de eLearning bazat pe blockchain trebuie să pornească de la cerințele reale ale produsului. Platforma gestionează utilizatori, autentificare, cursuri, conținut, evaluări, organizații, roluri, portofele, recompense și operații de audit. Aceste fluxuri cer consistență, securitate, performanță și posibilitatea de a evolua fără rescriere completă. Din acest motiv, criteriile de selecție folosite în lucrare sunt: siguranța și robustețea codului, suportul pentru operații asincrone, maturitatea ecosistemului, integrarea cu PostgreSQL, compatibilitatea cu stocare de obiecte, capacitatea de a interacționa cu Ethereum și ergonomia pentru dezvoltarea frontend-ului.

Un criteriu esențial este separarea responsabilităților. Tehnologiile nu au fost alese pentru a produce un sistem spectaculos din punct de vedere tehnologic, ci pentru a susține o arhitectură explicabilă. Backend-ul trebuie să fie sursa de adevăr pentru reguli și permisiuni, baza de date trebuie să păstreze istoricul operațional, worker-ul trebuie să proceseze sarcini costisitoare în afara cererilor HTTP, iar blockchain-ul trebuie să gestioneze componentele care beneficiază de audit public.

3.2 Rust pentru backend

Rust a fost ales pentru backend deoarece oferă un echilibru bun între performanță, siguranța memoriei și control asupra erorilor. Limbajul pune accent pe proprietate, împrumuturi și tipuri explicite,

cea ce reduce o categorie importantă de defecte întâlnite în aplicațiile server: acces invalid la memorie, date mutate accidental sau stări tratate incomplet [23, 8]. Pentru o platformă care lucrează cu autentificare, permisiuni și tranzacții financiare interne, aceste proprietăți sunt relevante.

O alternativă ar fi fost Node.js, care are un ecosistem foarte matur pentru aplicații web și permite dezvoltare rapidă. Totuși, pentru logica de domeniu cu multe tipuri, stări și erori, Rust oferă o disciplină mai mare la compilare. Python sau Ruby ar fi redus costul inițial de dezvoltare, dar ar fi cerut mai multă atenție la validarea tipurilor și la performanță în fluxuri concurente. Java sau Kotlin ar fi fost opțiuni solide pentru backend enterprise, însă Rust oferă o amprentă de runtime mai mică și o integrare foarte bună cu sisteme asincrone moderne.

3.3 Actix Web pentru API

Actix Web a fost ales ca framework HTTP deoarece este matur, performant și integrat cu ecosistemul asincron Rust. Documentația oficială îl prezintă ca framework web pentru servicii HTTP, rutare, extractori și middleware [31]. În RustLearn, Actix Web permite definirea unui API modular, cu rute grupate pe contexte: identitate, cursuri, organizații, recompense, portofele, raportare și operații. Middleware-ul este important deoarece autentificarea JWT și verificările de permisiuni trebuie aplicate consecvent pe rute.

O alternativă posibilă ar fi fost Axum, bazat pe ecosistemul Tower. Axum are o integrare elegantă cu tipurile Rust și este foarte potrivit pentru servicii moderne. Alegerea Actix Web este justificată prin maturitate, exemple existente și potrivirea cu structura proiectului. Pentru lucrare, diferența critică nu este un benchmark între framework-uri, ci posibilitatea de a exprima clar structuri HTTP, extractori, middleware și state partajat.

3.4 PostgreSQL și Diesel

PostgreSQL a fost ales ca bază de date principală deoarece platforma are nevoie de relații complexe, tranzacții, constrângeri, indexuri și interogări pentru rapoarte. Datele educaționale și financiare interne nu sunt documente izolate, ci entități conectate: utilizatori, roluri, organizații, cursuri, înscrieri, progres, evaluări, candidați la recompense, tranzacții interne și evenimente de audit. Documentația PostgreSQL evidențiază suportul pentru un model relațional complet, extensibil și matur [29].

Diesel a fost ales pentru accesul la date deoarece oferă interogări tipizate și migrații in-

tegrate în ecosistemul Rust [21]. Într-un proiect cu multe tabele și multe permisiuni, beneficiul tipurilor este semnificativ: erorile de nume de coloane, tipuri incompatibile sau rezultate așteptate incorect sunt detectate mai devreme. Diesel Async permite menținerea modelului tipizat și în operații asincrone, ceea ce se potrivește cu Actix Web.

O alternativă ar fi fost o bază NoSQL, de exemplu MongoDB. Aceasta ar fi simplificat stocarea unor documente flexibile, dar ar fi complicat integritatea relațiilor dintre utilizatori, roluri, cursuri și tranzacții. Pentru RustLearn, consistența este mai importantă decât flexibilitatea documentelor. O altă alternativă ar fi fost un ORM mai dinamic, dar Diesel este preferabil tocmai pentru că impune o relație clară între schemă și cod.

3.5 Stocare S3 și worker pentru conținut multimedia

Conținutul educațional include materiale text, fișiere și video. Baza de date nu este potrivită pentru stocarea fișierelor mari, iar serverul HTTP nu trebuie să proceseze video în interiorul unei cereri. Din acest motiv, RustLearn folosește stocare compatibilă S3 pentru obiecte și un worker separat pentru procesare. Amazon S3 este un model consacrat de stocare de obiecte, organizat prin bucket-uri și chei, potrivit pentru fișiere distribuite și scalabile [16]. În dezvoltarea locală, RustFS oferă o implementare compatibilă S3.

Worker-ul procesează sarcini precum transcodarea video, extragerea audio, retry-uri și indexarea depunerilor de tokenuri. Această separare este valoroasă deoarece cererile HTTP rămân rapide, iar operațiile costisitoare sunt executate asincron. Dacă transcodarea video ar fi făcută în request, utilizatorul ar aștepta inutil, iar serverul API ar fi expus la blocaje. Un worker dedicat permite limitarea concurenței, monitorizarea cozii și retry-uri controlate.

3.6 Next.js pentru frontend

Frontend-ul a fost realizat cu Next.js deoarece platforma are nevoie de rute clare, componente React, posibilitatea de a separa suprafețele pentru cursanți, profesori, organizații și administratori, precum și de integrare cu API-ul backend. Documentația Next.js evidențiază App Router ca mecanism de rutare bazat pe sistemul de fișiere și pe funcționalități moderne React [33]. În RustLearn, această abordare permite rute precum `/learn`, `/teach`, `/organizations`, `/admin`, `/wallet` și `/rewards`.

O alternativă ar fi fost o aplicație React simplă cu Vite. Aceasta ar fi fost suficientă pentru

o interfață mai mică, dar Next.js oferă structură de rutare și convenții utile pentru un produs cu multe ecrane. O altă alternativă ar fi fost Flutter pentru web, dar ecosistemul React/Next.js este mai potrivit pentru interfețe administrative web dense, formulare, tabele, filtre și integrare rapidă cu API-uri HTTP.

3.7 Ethereum, Solidity și OpenZeppelin

Componenta blockchain folosește Ethereum și contracte Solidity pentru LearnToken. Solidity este limbajul principal pentru contracte inteligente pe EVM [30], iar OpenZeppelin oferă implementări verificate pentru standarde precum ERC-20 [26]. LearnToken extinde ERC-20 cu opțiuni precum burn și permit, iar contractele auxiliare permit operații semnate și import de tokenuri.

Standardul EIP-712 definește semnarea datelor structurate, ceea ce permite utilizatorilor să semneze mesaje mai clare decât octeți opaci [22]. EIP-2612 extinde ERC-20 cu funcția `permit`, prin care o aprobare poate fi realizată prin semnătură, nu printr-o tranzacție separată [32]. Aceste standarde sunt importante pentru experiența utilizatorului, deoarece reduc numărul de pași on-chain și permit platformei să construiască fluxuri în care utilizatorul semnează o intenție, iar execuția poate fi finalizată controlat.

Blockchain-ul nu este folosit pentru toate datele. Notele, lecțiile, profilurile, permisiunile și rapoartele rămân în baza de date. Această decizie este justificată de cost, performanță și protecția datelor. Blockchain-ul este folosit pentru tokenuri, tranzacții și evenimente în care transparența și interoperabilitatea sunt avantaje reale. Astfel, arhitectura evită atât centralizarea completă a recompenselor, cât și descentralizarea inutilă a datelor educaționale.

3.8 Arhitectură modulară monolitică

RustLearn adoptă o arhitectură modulară monolitică. Sistemul rulează ca o aplicație backend principală, dar codul este separat în contexte: `http`, `application`, `domain`, `infra` și `bootstrap`. Această organizare păstrează simplitatea operațională a unui monolit, dar introduce limite interne clare. În sistemele distribuite, împărțirea prematură în servicii poate aduce costuri de comunicare, consistență și observabilitate [3]. Pentru un produs aflat în evoluție, monolitul modular este mai ușor de testat, migrat și refactorizat.

Valoarea acestei arhitecturi constă în faptul că domeniile sunt separate în cod înainte de a fi separate fizic. Dacă în viitor worker-ul de recompense, serviciul de conținut sau indexerul

blockchain trebuie extrase în servicii independente, limitele conceptuale există deja. Până atunci, tranzacțiile locale, migrațiile și testele rămân mai simple.

3.9 Alternative analizate pentru backend

Pentru backend au fost analizate mai multe familii tehnologice. Node.js ar fi oferit productivitate ridicată și un ecosistem foarte bogat pentru API-uri, autentificare și integrare web [2]. Totuși, sistemul RustLearn conține fluxuri în care stările și erorile trebuie modelate riguros: recompense, portofele, permisiuni și tranzacții. Într-un limbaj dinamic, aceste reguli pot fi implementate corect, dar o parte mai mare din verificare rămâne la testare și review. Rust transferă multe erori către compilare, iar acest lucru este valoros într-un domeniu cu consecințe financiare.

Java și Kotlin ar fi fost opțiuni solide pentru aplicații enterprise. Ele au ecosistem matur, instrumente de monitorizare și framework-uri stabile. Dezavantajul pentru această lucrare este costul de configurare și amprenta mai mare a runtime-ului. Rust oferă un executabil eficient, control explicit asupra concurenței și integrare bună cu biblioteci moderne de criptografie, Ethereum și procesare asincronă. Alegerea nu afirmă că Rust este universal superior, ci că se potrivește cu obiectivele specifice: siguranță, performanță și disciplină în modelarea domeniului.

Python sau Ruby ar fi permis prototipare rapidă. Pentru o demonstrație minimă, ele ar fi fost potrivite. Pentru o platformă cu autentificare, audit, recompense și worker, avantajul prototipării rapide se reduce deoarece fluxurile trebuie oricum testate riguros. În plus, integrarea cu contracte, semnături și tipuri financiare cere atenție la reprezentarea datelor. Rust nu elimină complexitatea, dar o face mai explicită.

3.10 Alternative analizate pentru frontend

Pentru frontend, o aplicație React cu Vite ar fi fost o variantă simplă și rapidă. Ea ar fi redus convențiile impuse de framework și ar fi oferit control direct asupra build-ului. Totuși, RustLearn are mai multe suprafețe distincte: cursant, profesor, organizație, administrator, portofel și recompense. Next.js oferă structură de rutare, organizare pe pagini și posibilitatea de a construi suprafețe dense fără a inventa manual toate convențiile proiectului [33].

Flutter pentru web ar fi permis o interfață unitară pentru mai multe platforme [11]. Totuși, produsul descris în lucrare este în primul rând o aplicație web administrativă și educațională, cu formulare, tabele, dashboard-uri, filtre și integrare HTTP. Ecosistemul React este mai natural pen-

tru acest tip de produs, iar integrarea cu biblioteci de UI, testare și rutare web este mai directă. De aceea, Next.js este justificat prin potrivirea cu fluxurile de produs, nu doar prin popularitate.

O altă opțiune ar fi fost separarea completă între aplicații: una pentru cursanți, una pentru profesori și una pentru administratori. Această separare poate fi utilă la scară mare, dar în faza unei lucrări aplicative ar crește costul de mentenanță. RustLearn păstrează o singură aplicație frontend, cu rute și componente diferențiate. Această alegere permite reutilizarea modelului de sesiune, a verificărilor de permisiuni și a componentelor pentru stări de încărcare, eroare și acces refuzat.

3.11 Contractul dintre straturi

Arhitectura modulară are valoare numai dacă limitele dintre straturi sunt respectate. În RustLearn, stratul HTTP nu ar trebui să decidă reguli de recompensare, iar stratul de infrastructură nu ar trebui să stabilească politici educaționale. Contractul dintre straturi este exprimat prin comenzi, porturi, DTO-uri și tipuri de domeniu. Handler-ul primește o cerere, o transformă într-o comandă și apelează un use case. Use case-ul verifică regulile și invocă porturi. Porturile sunt implementate în infrastructură.

Tabela 3.1. Contracte între straturile arhitecturii (elaborare proprie).

Limită	Ce trece prin limită	Ce nu trebuie să treacă
HTTP - aplicație	Comenzi validate, identitate actor, parametri de rută	Obiecte Actix sau detalii de serializare
Aplicație - domeniu	Tipuri de domeniu, statusuri, reguli pure	Pool PostgreSQL, request HTTP, JSON brut
Aplicație - infrastructură	Porturi, tranzacții, rezultate persistente	Decizii pedagogice ascunse în SQL
Infrastructură - servicii externe	API S3, Ethereum, email, fișiere temporare	Reguli de permisiuni sau politici de recompensare

Respectarea acestor limite face ca schimbările să fie localizate. Dacă formatul unui răspuns JSON se modifică, domeniul nu trebuie rescris. Dacă se schimbă furnizorul S3, use case-urile nu trebuie să cunoască detaliile noului SDK. Dacă o politică de recompensare se modifică, handler-ele nu trebuie să conțină ramuri noi. Această disciplină este una dintre justificările principale pentru structura aleasă.

3.12 Date off-chain și evenimente on-chain

O decizie importantă este delimitarea datelor care rămân off-chain de evenimentele care ajung on-chain. Datele personale, progresul detaliat, evaluările, permisiunile și notele profesorului trebuie să rămână off-chain. Ele sunt sensibile, se pot modifica, pot necesita corecturi și trebuie gestionate conform unor politici de confidențialitate. În schimb, transferul unui token, importul de fonduri sau arderea unui token pot fi reprezentate on-chain deoarece au valoare de audit public.

Tabela 3.2. Delimitarea informațiilor off-chain/on-chain (elaborare proprie).

Informație	Loc potrivit	Motiv
Profil utilizator, email, roluri	PostgreSQL	Date personale și operații administrative frecvente
Progres pe lecții și evaluări	PostgreSQL	Necesită actualizări dese și explicații contextuale
Politică de recompensare	PostgreSQL, cu audit	Reguli versionate, administrate de platformă
Transfer LearnToken	Ethereum și oglindire internă	Audit public, interoperabilitate și confirmare externă
Creditare wallet intern	PostgreSQL	Legătură operațională între recompensă și utilizator

Această delimitare evită două erori frecvente. Prima este supra-centralizarea, în care tokenul nu are valoare externă și rămâne doar o coloană în baza de date. A doua este supra-descentralizarea, în care platforma scrie prea multe informații pe blockchain și devine scumpă, lentă și greu de corectat. RustLearn folosește blockchain-ul ca strat de valoare, nu ca depozit general de date educaționale.

3.13 Costuri operaționale și costuri cognitive

Tehnologiile trebuie evaluate nu doar prin performanță, ci și prin costurile pe care le impun utilizatorilor și operatorilor. Blockchain-ul introduce costuri de gas, latență de confirmare, semnături și gestionarea portofelelor. Pentru un utilizator obișnuit cu aplicații web clasice, aceste concepte pot fi dificile. De aceea, interfața trebuie să ascundă detaliile inutile și să explice doar deciziile care

contează: ce semnez, ce primesc, cine plătește gas-ul și ce se întâmplă dacă tranzacția eșuează.

Costurile cognitive apar și pentru administratori. O platformă cu recompense trebuie să ofere dashboard-uri clare pentru candidați, politici, fraud blocks, wallet-uri și reconciliere. Dacă operatorul trebuie să inspecteze manual tabele sau loguri pentru fiecare incident, sistemul nu este operabil. De aceea, alegerea tehnologiilor este legată de modelul de administrare: PostgreSQL permite rapoarte și interogări, Diesel menține coerența tipurilor, iar Next.js permite construirea unor suprafețe dense de management.

3.14 De ce monolit modular, nu microservicii premature

Microserviciile sunt utile atunci când domeniile au ritmuri de scalare diferite, echipe separate și nevoi de deploy independent. În cazul RustLearn, o separare fizică timpurie ar adăuga probleme: comunicare între servicii, consistență distribuită, tracing, retry-uri, contracte de API interne și gestionarea mai multor procese. Pentru o disertație și pentru o platformă aflată în evoluție, aceste costuri ar masca problema principală: corectitudinea fluxurilor educaționale și de recompensare.

Monolitul modular păstrează simplitatea deploy-ului, dar nu abandonează disciplina. Modulele de domeniu, aplicație, infrastructură și HTTP sunt delimitate în cod. Dacă în viitor indexerul blockchain sau procesarea video trebuie scalate separat, worker-ul oferă deja un punct natural de extracție. Dacă raportarea devine costisitoare, poate fi mutată într-un serviciu dedicat. Arhitectura actuală pregătește aceste evoluții fără a introduce complexitate înainte ca ea să fie necesară.

3.15 Mentenabilitate și evoluție

Mentenabilitatea unei platforme educaționale depinde de capacitatea de a schimba reguli fără a produce regresii. Cursurile pot primi noi tipuri de conținut, evaluările pot introduce noi scoruri, organizațiile pot cere rapoarte suplimentare, iar recompensele pot avea politici noi. Dacă aceste schimbări sunt dispersate în handler-e, SQL brut și componente frontend fără convenții, dezvoltarea devine riscantă. Structura RustLearn reduce acest risc prin tipuri de domeniu, use case-uri, migrații și teste.

Evoluția tehnică este susținută și de faptul că deciziile importante sunt explicitate. Alegerea Rust nu este doar o preferință de limbaj, ci o alegere pentru siguranță. Alegerea PostgreSQL nu este doar comoditate, ci o alegere pentru integritatea relațiilor. Alegerea Ethereum nu este doar o asociere cu blockchain-ul, ci o alegere pentru tokenuri standardizate și interoperabile. O disertație

aplicativă trebuie să poată explica aceste decizii, deoarece valoarea ei nu stă doar în faptul că sistemul funcționează, ci în faptul că sistemul este justificat.

3.16 Securitatea prin alegerea tehnologiilor

Securitatea nu este obținută doar prin adăugarea unui modul de autentificare. Ea este influențată de fiecare alegere tehnologică. Rust contribuie prin sistemul de tipuri și prin siguranța memoriei. PostgreSQL contribuie prin tranzacții, constrângeri și integritate referențială. Diesel contribuie prin interogări tipizate și prin legarea codului de schema bazei de date. OpenZeppelin contribuie prin implementări verificate ale standardelor tokenizate. Next.js contribuie printr-o structură clară a suprafețelor frontend, iar Actix Web prin middleware și extractori care pot aplica reguli consecutive.

Această perspectivă este importantă deoarece o vulnerabilitate apare adesea la intersecția dintre straturi. O rută HTTP poate valida greșit actorul, o interogare poate omite filtrul de organizație, un frontend poate afișa o acțiune fără permisiune, iar un contract poate accepta o semnătură reutilizată. Alegerea tehnologiilor trebuie deci corelată cu modul în care ele reduc aceste erori. RustLearn nu se bazează pe o singură apărare, ci pe straturi: autentificare, autorizare, tipuri, tranzacții, audit, nonce-uri și reconciliere.

3.17 Portabilitate și independență operațională

O altă justificare a tehnologiilor este portabilitatea. PostgreSQL, S3, Docker Compose, Kubernetes și Ethereum sunt tehnologii care pot fi operate în medii diferite. În dezvoltare, S3 poate fi reprezentat de RustFS, iar Ethereum de Anvil. În producție, aceleași interfețe pot fi conectate la servicii administrate sau la infrastructură proprie. Această portabilitate reduce dependența de un singur furnizor și permite testarea locală a unor fluxuri care altfel ar cere servicii externe.

Portabilitatea nu înseamnă absența costurilor. Fiecare adaptor trebuie configurat, monitorizat și testat. Totuși, beneficiul este că arhitectura rămâne explicabilă: aplicația cunoaște un contract S3, nu un bucket legat ireversibil de un furnizor; aplicația cunoaște un contract EVM, nu o rețea unică; aplicația cunoaște PostgreSQL, nu o schemă ascunsă într-un serviciu proprietar. Pentru o lucrare de disertație, această independență este valoroasă deoarece demonstrează că sistemul poate fi înțeles și rulat în afara unui mediu opac.

3.18 Criterii de respingere a alternativelor

Alegerile tehnologice devin mai convingătoare atunci când sunt însoțite de criterii de respingere. O tehnologie a fost respinsă dacă introducea complexitate disproporționată, dacă muta prea multă logică în runtime, dacă făcea dificil auditul sau dacă nu răspundea direct cerințelor. De exemplu, o bază de date documentară ar fi fost flexibilă, dar ar fi slăbit relațiile dintre cursuri, roluri, organizații și tranzacții. O arhitectură de microservicii ar fi arătat sofisticat, dar ar fi complicat tranzacțiile și testarea. O soluție complet on-chain ar fi adus transparență, dar ar fi încălcat cerințe de confidențialitate și performanță.

Tabela 3.3. Criterii folosite pentru respingerea alternativelor (elaborare proprie).

Alternativă	Avantaj inițial	Motiv de respingere
NoSQL ca stocare principală	Flexibilitate de schemă	Relații și constrângeri mai greu de auditat
Microservicii timpurii	Deploy independent pe domenii	Cost operațional și consistență distribuită premature
Date educaționale on-chain	Transparență maximă	Cost, latență și expunere de date sensibile
Frontend-uri separate pe roluri	Izolare vizuală clară	Duplicare de sesiune, componente și logică de permisiuni
Token custom fără standarde	Control complet asupra ABI-ului	Risc crescut și interoperabilitate scăzută

3.19 Comparație sintetică

Tabelul 3.4 prezintă pe scurt tehnologiile folosite și motivele alegerii lor.

Tabela 3.4. Tehnologii folosite și justificarea alegerii (elaborare proprie).

Componentă	Tehnologie	Justificare
Backend API	Rust și Actix Web	Siguranță la compilare, performanță, middleware, rutare modulară și potrivire cu aplicații asincrone.
Persistentă	PostgreSQL și Diesel	Model relațional, tranzacții, migrații, tipare clare pentru date conectate și interogări tipizate.
Media	S3 compatibil și worker	Fișierele mari sunt separate de baza de date, iar procesarea video nu blochează cererile HTTP.
Frontend	Next.js și React	Rutare pe suprafețe de produs, componente reutilizabile și interfețe diferite pentru roluri diferite.
Blockchain	Solidity, Ethereum, OpenZeppelin	Token ERC-20, standarde mature, semnături EIP-712 și permit EIP-2612 pentru fluxuri Web3 controlate.
Operare	Docker Compose și Kubernetes	Aliniere între dezvoltare locală, testare și deploy, cu servicii pentru API, web, DB, S3, Anvil și worker.

3.20 Concluzie

Tehnologiile selectate susțin o arhitectură hibridă în care fiecare componentă are un rol clar. Rust și Actix Web oferă un backend robust, PostgreSQL și Diesel susțin consistența datelor, S3 și worker-ul gestionează conținutul multimedia, Next.js oferă suprafețe de produs moderne, iar Ethereum adaugă valoare acolo unde tokenizarea și auditul public contează. Alegerea nu urmărește maximizarea descentralizării, ci construirea unui sistem educațional util, sigur și extensibil.

Partea a II-a

Capitolul 4

Analiza și arhitectura sistemului RustLearn

4.1 Descriere generală

RustLearn este platforma aplicativă dezvoltată pentru tema *Sistem de eLearning bazat pe blockchain*. Sistemul combină funcții de eLearning cu mecanisme de recompensare bazate pe LearnToken. Din punct de vedere funcțional, platforma permite autentificarea utilizatorilor, administrarea cursurilor, publicarea conținutului, urmărirea progresului, gestionarea organizațiilor, evaluarea cursanților, administrarea portofelelor și operarea unui flux de recompense. Din punct de vedere tehnic, sistemul este format dintr-un backend Rust/Actix Web, o bază de date PostgreSQL, stocare compatibilă S3, un worker de procesare, contracte Solidity și un frontend Next.js.

Arhitectura a fost proiectată pentru a evita două extreme. Prima extremă este platforma complet centralizată, în care toate recompensele sunt doar înregistrări interne. Aceasta ar fi simplă, dar ar reduce valoarea de transparență și interoperabilitate a tokenului. A doua extremă este platforma complet on-chain, în care fiecare progres, rol sau evaluare ar fi tranzacție blockchain. Aceasta ar fi costisitoare, lentă și nepotrivită pentru date educaționale sensibile. RustLearn folosește o cale intermediară: operațiile educaționale rămân off-chain, iar tokenurile și tranzacțiile relevante sunt integrate cu Ethereum.

4.2 Actorii sistemului

Sistemul are patru categorii principale de actori. Cursantul consumă conținut, solicită înscrierea la cursuri, parcurge lecții, susține evaluări, urmărește recompensele și își gestionează portofelul. Profesorul creează conținut, administrează cursurile pe care le predă, verifică înscrieri, evaluează studenți și aprobă candidați la recompensă în limita permisiunilor primite. Organizația gestionează

programe educaționale, membri, cursuri sponsorizate, rapoarte și bugete de recompensare. Administratorul platformei gestionează roluri globale, aplicații de profesor, politici de recompensare, blocaje antifraudă, exporturi, KYC, portofele și reconciliere.

Acești actori nu sunt doar etichete de interfață. Ei corespund unor scopuri de permisiuni diferite: platformă, organizație și curs. Această separare este necesară deoarece același utilizator poate fi student într-un curs, profesor în alt curs și membru al unei organizații. Dacă sistemul ar folosi doar roluri globale, ar apărea fie privilegii excesive, fie restricții prea dure. Modelul pe scopuri permite exprimarea unei realități educaționale mai fine.

4.3 Cerințe funcționale

Cerințele funcționale principale sunt:

- autentificare cu token JWT, politici de parolă, verificare email și expunere JWKS pentru verificare externă;
- administrarea cursurilor, capitolelor, conținutului și stărilor de publicare;
- urmărirea progresului și susținerea evaluărilor cu scor și prag de promovare;
- gestionarea organizațiilor, membrilor, cursurilor asociate și rapoartelor;
- flux de aplicare și aprobare pentru profesori;
- portofele interne pentru utilizatori și organizații, cu audit al tranzacțiilor;
- candidați la recompensă, politici de recompensare, aprobări, plăți, creditare wallet și notificări;
- blocaje antifraudă și reconciliere pentru stări inconsistente;
- procesarea fișierelor video prin worker și stocare de obiecte;
- health checks și readiness checks pentru baza de date, S3 și Ethereum.

4.4 Cerințe nefuncționale

Cerințele nefuncționale sunt la fel de importante ca funcțiile vizibile. Platforma trebuie să fie sigură, deoarece gestionează identitate, portofele și permisiuni. Trebuie să fie auditabilă, deoarece

recompensele pot avea valoare economică și pot genera dispute. Trebuie să fie extensibilă, deoarece fluxurile educaționale și regulile de recompensare pot evolua. Trebuie să fie operabilă, adică să poată fi rulată local, testată, migrată și monitorizată. De asemenea, trebuie să fie rezilientă la eșecuri parțiale: o tranzacție blockchain eșuată nu trebuie să producă dublă creditare, iar un eșec de procesare video nu trebuie să blocheze API-ul.

4.5 Arhitectura de ansamblu

Figura 4.1 prezintă arhitectura de ansamblu. Browserul comunică cu aplicația Next.js, iar aceasta consumă API-ul Actix Web. Backend-ul aplică middleware de autentificare și permisiuni, apoi direcționează cererile către use case-uri. Use case-urile lucrează cu reguli de domeniu și adaptoare de infrastructură. PostgreSQL păstrează starea aplicației, S3 păstrează obiectele media, iar Ethereum păstrează tokenul și evenimentele on-chain. Worker-ul preia sarcini din bază de date și interacționează cu S3 și Ethereum fără a bloca cererile utilizatorilor.

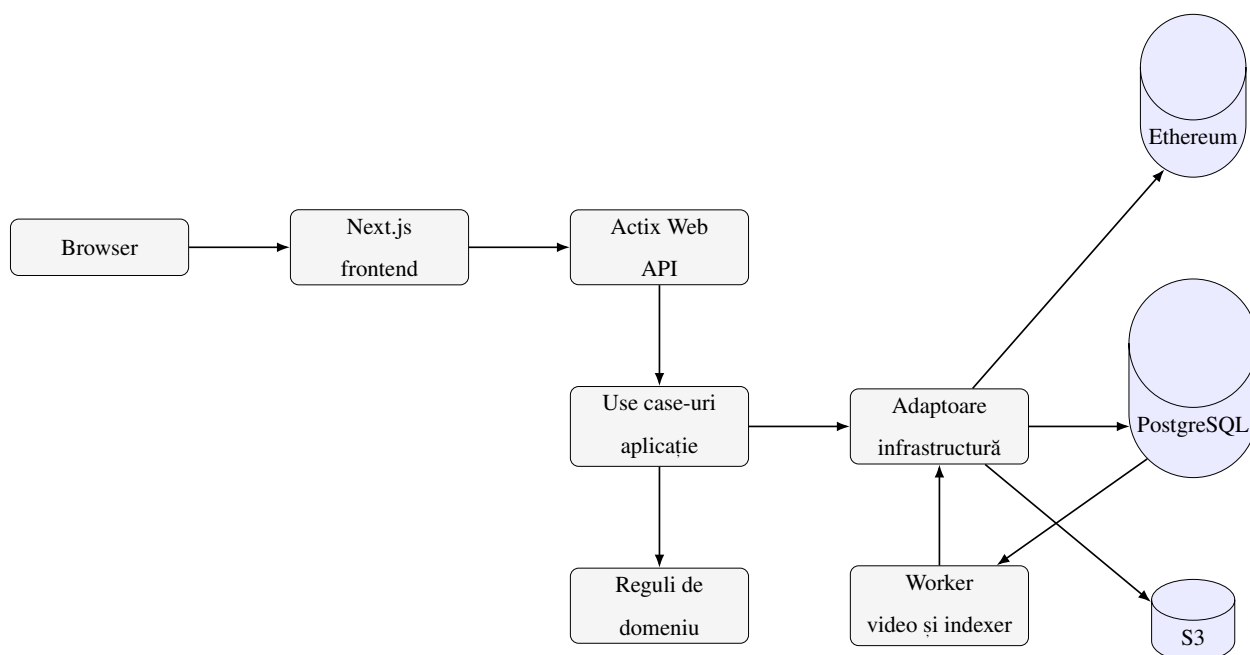


Figura 4.1. Arhitectura generală RustLearn (elaborare proprie).

4.6 Straturile aplicației

Codul backend este separat în straturi. Directorul `src/http` conține rute, DTO-uri, extractor și middleware. Directorul `src/application` conține use case-uri, comenzi, porturi și rezultate. Directorul `src/domain` păstrează vocabularul de domeniu și regulile pure. Directorul

`src/infra` conține adaptoare concrete pentru PostgreSQL, S3, Ethereum, JWT și notificări. Directorul `src/bootstrap` compune aplicația, configurează starea, înregistrează rutele și pornește contractele necesare.

Această separare este valoroasă deoarece împiedică amestecarea logicii de business cu detaliile HTTP. De exemplu, o regulă privind validarea unei recompense nu trebuie să depindă de un handler Actix. Ea trebuie să poată fi testată ca serviciu de aplicație sau regulă de domeniu. La fel, interogările PostgreSQL trebuie încapsulate în adaptoare, nu scrise direct în rute. Rezultatul este un cod mai testabil și mai ușor de modificat.

Tabela 4.1. Responsabilitățile straturilor backend (elaborare proprie).

Strat	Responsabilitate
<code>http</code>	Expune API-ul, validează forma cererilor, aplică middleware și transformă răspunsurile în JSON.
<code>application</code>	Ordonează pașii unui caz de utilizare, verifică porturi, produce rezultate și erori controlate.
<code>domain</code>	Definește reguli și concepte independente de framework: roluri, permisiuni, statusuri, tipuri de recompensă.
<code>infra</code>	Implementează accesul concret la PostgreSQL, S3, Ethereum, JWT, email local și notificări.
<code>bootstrap</code>	Leagă modulele, construiește starea aplicației și configurează pornirea procesului.

4.7 Modelul de permisiuni

Modelul de permisiuni este unul dintre elementele care dau valoare arhitecturii. RustLearn nu se limitează la roluri globale. Există roluri de platformă, roluri de organizație și roluri de curs. De exemplu, un utilizator poate avea permisiunea `VIEW_COURSE` într-un curs, `MANAGE_ORG_MEMBERS` într-o organizație și nicio permisiune administrativă la nivel de platformă. Middleware-urile specializate verifică aceste scopuri înainte ca operațiile sensibile să fie executate.

Figura 4.2 prezintă modelul conceptual. Permisia efectivă este rezultatul combinației dintre identitatea utilizatorului, rolurile directe, rolurile pe organizație, rolurile pe curs și eventualele permisiuni delegate. Acest model permite administrare fină și reduce riscul ca un actor să

primească privilegii mai largi decât sunt necesare.

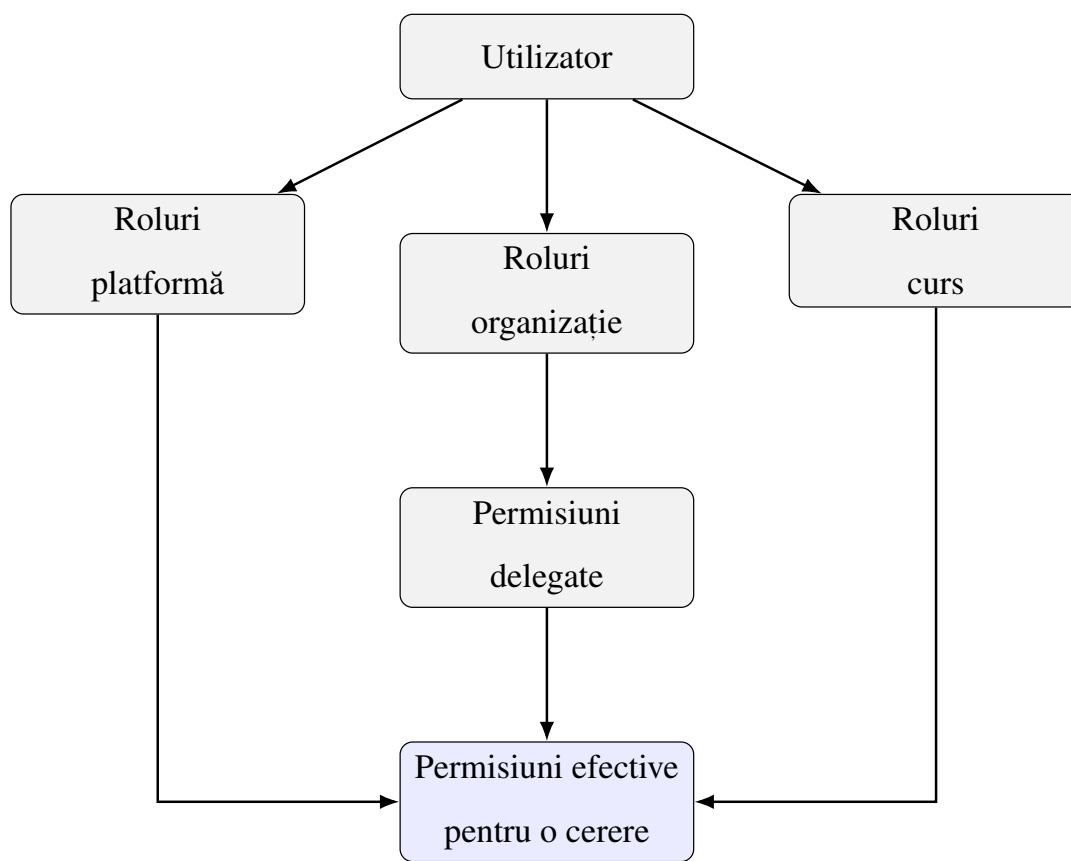


Figura 4.2. Model conceptual al permisiunilor pe scopuri (elaborare proprie).

4.8 Fluxul de recompensare

Fluxul de recompensare este proiectat ca un proces auditat. Un eveniment educațional creează un candidat la recompensă. Profesorul sau sistemul verifică evenimentul, platforma poate aproba suma, apoi se planifică plata, se înregistrează confirmarea tokenului, se creditează portofelul intern, se trimite notificarea și se rulează reconcilierea. Această succesiune previne dublarea recompenselor și permite intervenție umană atunci când apar erori sau suspiciuni.

Figura 4.3 prezintă stările principale ale fluxului. Unele etape pot fi automate, iar altele pot necesita aprobare. Valoarea arhitecturii constă în faptul că fiecare tranziție are un motiv și poate fi auditată. Dacă tranzacția blockchain este confirmată, dar creditarea portofelului intern eșuează, sistemul nu trimite încă o tranzacție. Reconcilierea pornește de la starea confirmată și completează etapa lipsă.

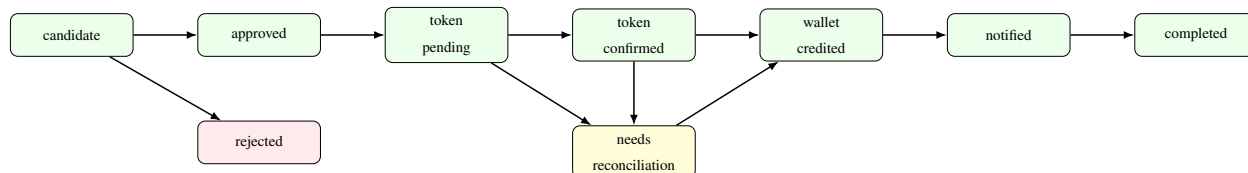


Figura 4.3. Fluxul de stări pentru recompensa LearnToken (elaborare proprie).

4.9 Persistența datelor

Schema PostgreSQL include tabele pentru utilizatori, autentificări, cursuri, capitole, conținut, progres, evaluări, organizații, roluri, permisiuni, notificări, portofele, tranzacții interne și externe, candidați la recompense, politici de recompensare, audit, fraud blocks, compensări și token burns. Această structură arată că platforma nu tratează recompensa ca o coloană simplă atașată cursului, ci ca un domeniu de sine stătător.

Persistența are două roluri. Primul este rolul operațional: aplicația trebuie să știe ce cursuri există, cine are acces, ce conținut este publicat și ce recompense sunt în așteptare. Al doilea este rolul de audit: platforma trebuie să poată explica de ce o recompensă a fost aprobată, cine a aprobat-o, ce politică era activă, ce tranzacție a fost creată și dacă portofelul a fost creditat. Pentru un sistem cu componente blockchain, baza de date nu concurează cu blockchain-ul, ci îl completează.

4.10 Use case-uri critice

Analiza sistemului poate fi clarificată prin descrierea use case-urilor critice. Primul use case este înscrierea și parcurgerea unui curs. Cursantul descoperă un curs publicat, solicită înscrierea atunci când este necesar, primește acces, parcurge conținutul și își vede progresul. Al doilea use case este autoratul de curs. Profesorul creează cursul, adaugă capitole, publică materiale, definește evaluări și gestionează cererile de înscriere. Al treilea use case este recompensarea. Un eveniment educațional produce un candidat, profesorul confirmă meritul, platforma decide suma, iar sistemul execută plata și creditarea.

Al patrulea use case este administrarea organizațională. O organizație poate grupa membri, cursuri și programe de învățare. Administratorul organizației are nevoie de rapoarte despre participare, progres, recompense sponsorizate și portofel. Al cincilea use case este operarea platformei. Administratorul global gestionează politici de recompensare, fraud blocks, aplicații de profesor, KYC, exporturi și stări de sistem. Aceste use case-uri arată de ce aplicația nu poate fi modelată ca

simplicu catalog de cursuri.

Tabela 4.2. Use case-uri critice și date afectate (elaborare proprie).

Use case	Actor principal	Date sau stări afectate
Parcurgere curs	Cursant	Înscriere, progres, evaluări, dashboard cursant
Autorat curs	Profesor	Cursuri, capitole, conținut, publicare, roster
Recompensare	Profesor și platformă	Candidați, politici, audit, tranzacții, wallet
Program organizațional	Administrator organizație	Membri, roluri, cursuri asociate, rapoarte
Operare platformă	Administrator platformă	Fraud blocks, KYC, exporturi, readiness, politici

4.11 Fluxul de finalizare a unui curs recompensat

Fluxul de finalizare a unui curs recompensat este reprezentativ pentru arhitectură deoarece traversează mai multe domenii. Cursantul finalizează lecțiile și promovează evaluarea. Domeniul de învățare înregistrează progresul și determină dacă termenii de finalizare sunt îndepliniți. Domeniul de recompense primește un eveniment eligibil și creează un candidat idempotent. Profesorul poate confirma sau respinge candidatul, iar platforma poate aproba suma. Abia apoi se planifică plata și se leagă tranzacția tokenizată de wallet.

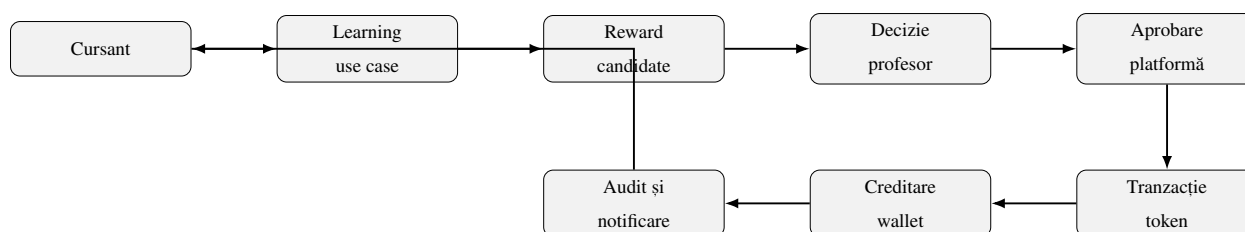


Figura 4.4. Fluxul finalizării unui curs recompensat (elaborare proprie).

Acest flux demonstrează de ce recompensa nu poate fi un efect secundar ascuns într-o rută de progres. Dacă un utilizator repetă aceeași evaluare, sistemul trebuie să recunoască evenimentul deja procesat. Dacă profesorul respinge candidatul, trebuie păstrat motivul. Dacă tranzacția token

este confirmată, dar creditarea internă eșuează, reconcilierea trebuie să repare doar etapa lipsă. Fiecare tranziție are deci o justificare tehnică și una de produs.

4.12 Modelarea stărilor și invariantelor

Platforma conține mai multe cicluri de viață. Cursurile pot fi draft, submitted, published sau archived. Aplicațiile de profesor pot fi pending, approved, rejected sau returned pentru completări. Candidații la recompense parcurg stări de validare și execuție. Joburile worker-ului pot fi queued, processing, completed sau failed. Fiecare ciclu de viață impune invariante: un curs nepublicat nu trebuie să fie vizibil public, o recompensă respinsă nu trebuie plătită, un job finalizat nu trebuie procesat din nou fără motiv administrativ.

Aceste invariante sunt mai ușor de controlat atunci când stările sunt explicite. O coloană booleană precum `paid=true` nu ar fi suficientă pentru recompense, deoarece nu distinge între plată planificată, tranzacție confirmată și wallet creditat. Prin urmare, RustLearn folosește stări descriptive și evenimente de audit. În plus, baza de date poate impune unicitate și referințe, iar codul de domeniu poate valida tranzițiile permise.

Tabela 4.3. Exemple de invariante arhitecturale (elaborare proprie).

Domeniu	Invariantă	Mecanism
Cursuri	Conținutul draft nu este vizibil cursanților neautorizați	Stări de publicare și permisiuni pe curs
Recompense	Același eveniment nu produce două recompense valide	Cheie de idempotentă și audit
Wallet	Creditarea internă urmează confirmării tokenului	Tranziții de stare și reconciliere
Permiuni	Rolurile sunt evaluate în scopul resursei accesate	Middleware pe platformă, organizație și curs
Worker	Un job eșuat nu blochează permanent coada	Retry, status și heartbeat

4.13 Frontiere între domenii

Un risc al aplicațiilor mari este amestecarea domeniilor. De exemplu, domeniul de învățare poate fi tentat să modifice direct portofelul atunci când un curs este finalizat. Această soluție pare rapidă, dar creează dependențe greu de urmărit. În RustLearn, domeniul de învățare produce evenimente eligibile, iar domeniul de recompense decide dacă ele devin candidați. Portofelul este creditat doar după ce recompensa a trecut prin pașii de aprobare și confirmare.

Această frontieră face sistemul mai ușor de explicat. Profesorul poate vedea progresul și candidații la recompense, dar nu trebuie să manipuleze direct ledger-ul intern. Administratorul platformei poate gestiona politici și blocaje, dar nu trebuie să modifice conținutul cursului. Worker-ul poate procesa video și indexa evenimente, dar nu decide reguli pedagogice. Fiecare domeniu are un vocabular propriu, iar interacțiunile dintre domenii sunt controlate prin use case-uri.

4.14 Arhitectura rapoartelor și exporturilor

Rapoartele sunt importante pentru organizații și administratori deoarece transformă datele operaționale în decizii. Un administrator de organizație trebuie să poată vedea câți membri sunt activi, ce cursuri sunt asociate organizației, ce recompense au fost sponsorizate și ce sold există în wallet. Administratorul platformei are nevoie de o vedere mai largă: candidați în așteptare, blocaje antifraudă active, tranzacții eșuate, exporturi CSV și stări de readiness.

Din punct de vedere arhitectural, raportarea nu trebuie să fie doar o interogare ad-hoc scrisă în interfață. Ea trebuie să respecte permisiunile și să diferențieze datele pe scopuri. Un membru obișnuit al unei organizații nu trebuie să vadă informații financiare sensibile. Un profesor nu trebuie să vadă rapoarte globale ale platformei. De aceea, reporting-ul folosește aceleași principii ca restul aplicației: use case-uri, permisiuni și DTO-uri explicite.

4.15 Valoarea pentru fiecare actor

Pentru cursant, valoarea arhitecturii constă în claritate și încredere. El vede progresul, recompensele și stările wallet-ului fără a fi obligat să înțeleagă detaliile infrastructurii. Pentru profesor, valoarea constă în controlul conținutului și al candidaților la recompense. Profesorul poate administra cursul, dar nu poate încălca regulile globale de plată. Pentru organizație, valoarea constă în posibilitatea de a sponsoriza învățarea și de a urmări rezultatele. Pentru platformă, valoarea constă

în audit, antifraudă și operabilitate.

Această analiză pe actori arată că arhitectura nu este o simplă diagramă tehnică. Ea răspunde unor tensiuni reale: autonomie versus control, recompensă versus fraudă, transparență versus confidențialitate, viteză de dezvoltare versus mentenanță. O soluție valoroasă nu elimină aceste tensiuni, ci le face explicite și le administrează prin limite clare.

4.16 Scalabilitate conceptuală

Scalabilitatea nu înseamnă doar mai multe cereri pe secundă. Într-o platformă educațională, scalabilitatea conceptuală înseamnă capacitatea de a adăuga noi tipuri de cursuri, noi reguli de recompensare, noi roluri și noi integrări fără a schimba fundamentul. RustLearn susține această evoluție prin separarea domeniilor și prin modelarea explicită a stărilor. De exemplu, un nou tip de recompensă poate porni de la același model de candidat, politică, aprobare și execuție.

La nivel tehnic, scalabilitatea poate fi abordată incremental. API-ul poate fi replicat, PostgreSQL poate fi optimizat prin indexuri și conexiuni gestionate, S3 poate livra fișiere mari independent de baza de date, iar worker-ul poate fi multiplicat dacă joburile sunt idempotente. Blockchain-ul rămâne folosit selectiv, ceea ce limitează costurile. Acest mod de scalare este realist pentru o platformă în creștere, deoarece nu presupune trecerea imediată la o arhitectură distribuită complexă.

4.17 Analiza cazurilor de eroare

O arhitectură valoroasă nu descrie doar fluxul fericit, ci și cazurile de eroare. În RustLearn, erorile posibile sunt tratate ca stări sau rezultate controlate. Dacă autentificarea eșuează, API-ul returnează un răspuns clar și nu execută handler-ul sensibil. Dacă utilizatorul nu are permisiune, middleware-ul sau use case-ul oprește operația. Dacă o recompensă este blocată de o politică antifraudă, candidatul rămâne explicabil, nu dispare. Dacă worker-ul nu poate procesa un video, jobul capătă stare de eșec și mesajul poate fi afișat profesorului.

Integrarea blockchain introduce erori speciale. O tranzacție poate fi respinsă, poate rămâne în așteptare, poate fi confirmată după o întârziere sau poate fi observată de indexer după restart. Arhitectura răspunde prin separarea etapelor: planificare, confirmare, creditare, notificare și reconciliere. Această separare este mai lungă decât un update direct, dar permite repararea controlată. Într-un sistem cu valoare economică, claritatea stărilor este mai importantă decât scurtarea fluxului.

Tabela 4.4. Cazuri de eroare și răspuns arhitectural (elaborare proprie).

Caz de eroare	Efect posibil	Răspuns arhitectural
Cerere fără JWT valid	Acces la rută protejată	Middleware de autentificare
Rol insuficient în curs	Modificare neautorizată	Permițiuni pe scop și use case-uri
Video invalid sau S3 indisponibil	Material neprocesat	Job cu status, retry și notificare
Tranzacție token întârziată	Wallet necreditat imediat	Stare intermediară și reconciliere
Fraud block activ	Recompensă nejustificată	Oprire flux și audit al blocului
Migrație parțială	Inconsistență date	Make targets, backup și verificări schema

4.18 Decizii privind granularitatea auditului

Auditul poate fi prea sărac sau prea zgomotos. Dacă se auditează doar tranzacția finală, se pierde explicația deciziei. Dacă se auditează fiecare citire minoră, istoricul devine greu de folosit. RustLearn adoptă o granularitate orientată pe decizii: aplicarea pentru profesor, aprobarea sau respingerea, schimbarea politicii de recompensare, crearea unui fraud block, decizia asupra candidatului, confirmarea tokenului și creditarea wallet-ului. Aceste evenimente au semnificație de business și pot fi utile într-o dispută.

Granularitatea auditului este legată de arhitectură. Dacă logica ar fi împrăștiată în handler-e, auditul ar fi inconsistent. Pentru că deciziile trec prin use case-uri, fiecare tranziție importantă poate produce un eveniment uniform. Această uniformitate permite interfețe de audit, exporturi și rapoarte. Ea este, de asemenea, o formă de documentare runtime: istoricul sistemului explică nu doar ce date există, ci cum au ajuns în acea stare.

4.19 Valoarea arhitecturii

Arhitectura este valoroasă prin patru proprietăți. Prima este claritatea: fiecare strat are responsabilitate distinctă, iar fluxurile de business pot fi urmărite din HTTP către use case, domeniu și

infrastructură. A doua este controlul: permisiunile pe scopuri și auditul permit administrare fină. A treia este reziliența: worker-ul, coada de joburi și reconcilierea reduc impactul eșecurilor parțiale. A patra este extensibilitatea: platforma poate adăuga noi tipuri de recompense, noi rute frontend, noi reguli de curs sau noi contracte fără a schimba fundamentul.

Valoarea practică apare mai ales în fluxurile în care educația și tokenizarea se întâlnesc. O finalizare de curs nu devine automat transfer on-chain; ea devine o decizie verificabilă. Un token nu este doar un număr într-o bază de date; el are o reprezentare prin contract. O eroare nu este doar un mesaj pierdut; ea produce o stare ce poate fi reconciliată. Această combinație face sistemul mai potrivit pentru o platformă educațională reală decât o arhitectură simplificată în care toate aceste operații sunt tratate informal.

4.20 Concluzie

RustLearn folosește o arhitectură hibridă și modulară pentru a susține cerințe educaționale și blockchain în același sistem. Backend-ul rămâne responsabil pentru reguli, permisiuni și fluxuri, baza de date păstrează starea și auditul, worker-ul procesează activități asincrone, iar Ethereum adaugă transparență pentru tokenuri. Această arhitectură oferă un echilibru între performanță, control și verificabilitate.

Capitolul 5

Implementarea platformei

5.1 Pornirea aplicației și compunerea stării

Implementarea backend-ului pornește de la un entrypoint subțire. Fișierul `src/main.rs` inițializează variabilele de mediu, logging-ul, starea aplicației și serverul Actix Web. Detaliile concrete de conectare la baza de date, S3, Ethereum și use case-uri sunt mutate în stratul `bootstrap`. Această decizie menține punctul de intrare ușor de citit și face ca inițializarea să poată fi testată sau refactorizată fără a modifica rutele HTTP.

Listing 5.1: Structura simplificată a pornirii serverului RustLearn.

```
let app_state = bootstrap::startup::initialize_app_state().await?;
HttpServer::new(move || {
    App::new()
        .configure(|cfg| bootstrap::app_data::configure_app_data(cfg, &
            app_state))
        .configure(bootstrap::routes::configure_routes)
})
.bind("0.0.0.0:8080")?
.run()
.await
```

Starea aplicației include pool-ul PostgreSQL, starea S3, use case-urile pentru controlul accesului, conținut, identitate, KYC, învățare, notificări, organizații, raportare, recompense, aplicații de profesor și portofele. Această compunere este importantă pentru că API-ul nu creează dependențe ad-hoc în fiecare handler. În schimb, handler-ul primește dependența deja configurată, iar testarea poate înlocui porturile cu implementări controlate.

5.2 API-ul și middleware-ul

API-ul este expus sub `/api`. Scope-ul principal este protejat prin middleware JWT și middleware condițional. Rutele sunt configurate pe module: identitate, notificări, învățare, organizații, raportare, KYC, recompense, control acces, aplicații profesor și portofele. Această organizare reflectă domeniile sistemului și permite adăugarea de funcții noi fără a transforma fișierul principal de rute într-o listă greu de administrat.

Autentificarea cu JWT permite cererilor să transporte identitatea utilizatorului. Verificările de permisiuni sunt aplicate ulterior, în funcție de resursa accesată. De exemplu, o rută de recompense la nivel de curs trebuie să verifice permisiuni de curs, iar o rută de fraud block la nivel de platformă trebuie să verifice permisiuni de platformă. Această diferențiere este importantă deoarece același utilizator poate avea autoritate într-un context și niciun drept în altul.

5.3 Identitate, parole și verificare email

Fluxurile de identitate includ înregistrarea, autentificarea, politica de parolă, verificarea emailului, resetarea parolei și expunerea cheii publice prin JWKS. Politica de parolă solicită lungime minimă și diversitate de caractere, ceea ce reduce riscul unor parole triviale. Emailul de verificare este în prezent simulat local prin mesaje de dezvoltare, dar modelul aplicației separă deja generarea tokenului de transportul efectiv, permițând integrarea viitoare a unui furnizor real de email.

Expunerea JWKS este utilă pentru verificarea externă a tokenurilor. În loc ca toate componentele să cunoască cheia privată, aplicația poate publica cheia publică pentru validare. Cheile private rămân în variabile de mediu și nu sunt comise în repository. Această separare respectă principiul că secretele operaționale nu trebuie să fie parte din codul sursă.

5.4 Modelul cursurilor și al progresului

Domeniul de învățare include cursuri, capitole, conținut, progres și evaluări. Cursurile au stare de ciclu de viață, descriere, subiecte și prerechizite. Conținutul are stare de publicare, iar progresul este înregistrat pentru relația dintre utilizator, curs și material. Evaluările includ întrebări, punctaj, încercări și prag de promovare. Această structură permite platformei să distingă între simpla vizualizare a unui material și promovarea unei evaluări.

Pentru recompense, evaluarea produce un handoff către domeniul rewards numai dacă re-

zultatul este relevant. Dacă o încercare nu este promovată, nu se creează candidat la recompensă. Dacă este promovată, sistemul poate genera un eveniment de tip `assessment_completion`. Această abordare păstrează clară relația dintre performanța educațională și recompensa potențială.

5.5 Organizații și aplicații de profesor

Organizațiile permit gestionarea unor programe educaționale cu membri, cursuri și rapoarte proprii. Un utilizator poate fi invitat, poate primi roluri, poate accesa cursuri sponsorizate și poate vedea rapoarte în funcție de permisiuni. Aplicațiile de profesor sunt tratate ca un flux separat: utilizatorul depune cerere, organizația poate susține cererea, iar platforma poate aproba, respinge sau cere modificări. Această etapizare previne acordarea automată a drepturilor de creare și recompensare a cursurilor.

Implementarea include audit pentru deciziile privind aplicațiile de profesor. Fiecare schimbare de status poate fi urmărită, iar interfața frontend poate afișa istoricul. Această trasabilitate este importantă deoarece profesorii pot influența direct conținutul și pot aproba candidați la recompense.

5.6 Recompense și politici

Subdomeniul rewards este împărțit în use case-uri. Există module pentru trimiterea unui candidat, decizia profesorului, aprobarea sumei, planificarea plății, confirmarea tokenului, creditarea portofelului, notificarea utilizatorului, reconciliere, compensări, politici și blocaje antifraudă. Această granularitate face fluxul mai lung, dar mai sigur. Într-un sistem cu valoare economică, o implementare scurtă și implicită ar fi mai riscantă decât un proces auditat.

Politicile de recompensare definesc evenimente eligibile, sume și stări de activare. Ele permit schimbarea regulilor fără a modifica istoricul deciziilor vechi. Dacă o politică este modificată, auditul poate indica versiunea folosită la momentul aprobării. În plus, blocajele antifraudă pot opri recompense pentru profesori, organizații, cursuri sau politici suspecte. Această funcție este necesară pentru a evita ca tokenizarea să creeze un stimulent pentru abuz.

5.7 Portofele și tranzacții

Portofelele interne permit urmărirea valorii asociate unui utilizator sau unei organizații. Sistemul diferențiază între tranzacții interne, tranzacții externe și evenimente blockchain. Tranzacția internă este utilă pentru ledger-ul aplicației, iar tranzacția externă păstrează informații precum chain id, adresă contract, hash de tranzacție, adresă sursă și destinație. Prin această separare, platforma poate explica legătura dintre starea internă și evenimentele on-chain.

Fluxurile de depunere și retragere pot avea costuri de gas plătite de utilizator sau de platformă. Pentru scenariile în care platforma plătește gas, sistemul poate aplica o taxă configurabilă. Acest detaliu arată că integrarea blockchain nu este doar tehnică, ci și economică. Platforma trebuie să știe cine suportă costul operației și cum este înregistrată această decizie.

5.8 Contractele inteligente

Contractul `LearnToken` extinde ERC-20, suportă burn și permit și permite mint doar de către owner. Această regulă reduce riscul emiterii necontrolate. `LearnTokenPresigner` permite depunerea de tokenuri și execuția unor transferuri semnate, cu nonce și deadline pentru protecție împotriva replay. `PlatformImporter` permite importul tokenurilor prin permit EIP-2612, transferând fonduri către treasury sau către un destinatar specific.

Listing 5.2: Fragment conceptual al transferului semnat.

```
require(nonces[signer] == nonce, "invalid nonce");
bytes32 digest = _hashTypedDataV4(structHash);
address recovered = ECDSA.recover(digest, signature);
require(recovered == signer, "invalid signature");
nonces[signer] = nonce + 1;
```

Această logică este importantă pentru securitate. Nonce-ul împiedică reutilizarea aceleiași semnături, deadline-ul limitează durata de valabilitate, iar semnătura EIP-712 face cererea mai explicită pentru utilizator. Contractele nu rezolvă singure toate problemele de business, dar oferă o bază verificabilă pentru operațiile cu tokenuri.

5.9 Worker, procesare video și indexare

Worker-ul rulează separat de API. El preia joburi, procesează încărcări video, execută retry-uri și scrie heartbeat pentru health check. Pentru video, fluxul descarcă fișierul din S3, rulează `ffmpeg`, generează un MP4 procesat și un fișier audio, apoi încarcă rezultatele înapoi în bucket. Utilizatorul primește notificare de succes sau eșec. Această abordare respectă principiul că operațiile lungi trebuie mutate din calea critică a request-ului.

Worker-ul include și indexer pentru depuneri de tokenuri. El citește blocuri Ethereum, filtrează evenimentele relevante, evită dublarea evenimentelor importate și actualizează starea persistentă cu următorul bloc de scanat. Astfel, sistemul poate sincroniza portofelul intern cu transferuri on-chain observate, fără a cere utilizatorului să repete acțiuni manuale.

5.10 Frontend-ul produsului

Frontend-ul Next.js este organizat în suprafețe de produs. `/learn` este dashboard-ul cursantului, `/courses` afișează catalogul, `/rewards` prezintă istoricul recompenselor, iar `/wallet` gestionează portofelul. Suprafețele `/teach` permit profesorilor să gestioneze cursuri, conținut, înscrieri, studenți și recompense. Rutele `/organizations` oferă administrare pentru organizații, iar `/admin` acoperă operațiile de platformă.

Interfața derivă acțiunile disponibile din permisiuni, nu doar din numele rolului. Acest lucru este important pentru că rolurile pot fi directe sau delegate, iar un utilizator poate avea acces limitat într-un singur context. Un buton administrativ nu trebuie afișat doar pentru că utilizatorul are un rol textual, ci pentru că sesiunea curentă conține permisiunea necesară. Această disciplină reduce confuzia și previne apelarea unor operații care ar fi oricum respinse de backend.

5.11 Notificări și feedback operațional

Notificările oferă utilizatorului feedback pentru evenimente importante: procesare video, creditare wallet, recompense sau operații administrative. Într-o platformă educațională, notificările nu sunt doar elemente de confort. Ele reduc incertitudinea. Dacă o recompensă este întârziată, utilizatorul trebuie să vadă starea. Dacă un video nu poate fi procesat, profesorul trebuie să primească un mesaj clar. Dacă o aplicație de profesor este respinsă, auditul trebuie să păstreze motivul.

5.12 Implementarea use case-urilor de aplicație

Use case-urile reprezintă centrul implementării. Ele primesc comenzi clare și returnează rezultate sau erori controlate. De exemplu, un use case pentru aprobarea unui candidat la recompensă nu trebuie să primească un request HTTP brut, ci actorul, identificatorul candidatului, decizia și motivul. Astfel, aceeași logică poate fi apelată dintr-o rută HTTP, dintr-un test de integrare sau dintr-un job administrativ. Această separare scade dependența de framework și crește capacitatea de testare.

Un model important este injectarea porturilor. Use case-ul nu cunoaște detalii despre Diesel sau Ethereum, ci lucrează cu interfețe care exprimă nevoi: găsirea candidatului, salvarea auditului, verificarea permisiunilor, crearea unei tranzacții sau trimiterea unei notificări. Implementările concrete sunt furnizate de stratul de infrastructură. Pentru o disertație, acest model arată maturitate arhitecturală: codul nu este doar funcțional, ci organizat pentru evoluție.

Listing 5.3: Structura conceptuală a unui use case.

```
pub async fn handle(command: ApproveRewardCommand) -> Result<RewardOutput,
    RewardError> {
    access_control.ensure_can_review(command.actor, command.course_id).await?;
    let candidate = rewards.find_candidate(command.candidate_id).await?;
    let decision = domain::approve_candidate(candidate, command.reason)?;
    rewards.save_decision(decision).await?;
    notifications.reward_decision(command.actor, command.candidate_id).await?;
    Ok(RewardOutput::approved(command.candidate_id))
}
```

Fragmentul este conceptual, dar descrie stilul implementării: verificare de acces, încărcare stare, regulă de domeniu, persistență, efect secundar controlat și răspuns. Această ordine reduce riscul ca sistemul să execute efecte externe înainte de a valida regulile principale. În fluxurile reale, unele operații sunt încapsulate în tranzacții, iar efectele externe sunt mutate în joburi sau etape ulterioare.

5.13 Idempotentă și consistență

Idempotentă este o proprietate critică pentru operațiile care pot fi repetate de utilizator, de browser, de un retry HTTP sau de un worker. Dacă un cursant apasă de două ori pe trimiterea evaluării, sistemul nu trebuie să creeze două recompense pentru același eveniment. Dacă un worker reia un job după eșec, nu trebuie să crediteze wallet-ul de două ori. Dacă un indexer vede același

eveniment blockchain după restart, trebuie să îl recunoască drept deja procesat.

RustLearn gestionează idempotenta prin chei unice, statusuri și tranziții controlate. Cheia de idempotentă poate include actorul, sursa, evenimentul și resursa. Pentru recompense, acest lucru înseamnă că același eveniment educațional produce cel mult un candidat valid. Pentru depuneri de tokenuri, evenimentul on-chain este identificat prin chain id, contract, hash, index de log și adrese relevante. Astfel, sistemul poate relua procesarea fără a produce efecte duplicate.

Consistența trebuie tratată diferit pentru operațiile locale și cele externe. În PostgreSQL, o tranzacție poate proteja mai multe scrieri. În Ethereum, confirmarea este externă și poate întârzia. De aceea, aplicația nu presupune că totul se finalizează într-un singur pas. Ea creează stări intermediare și mecanisme de reconciliere. Această abordare este mai realistă decât încercarea de a transforma blockchain-ul într-o extensie atomică a bazei de date.

5.14 Implementarea permisiunilor pe scopuri

Permisunile sunt implementate pe mai multe niveluri. La nivel de platformă, ele controlează operații precum aprobarea aplicațiilor de profesor, gestionarea politicilor, fraud blocks și exporturile globale. La nivel de organizație, ele controlează membrii, cursurile sponsorizate și rapoartele organizației. La nivel de curs, ele controlează autoratul, publicarea, înscrierile, studenții și candidații la recompense. Această structură corespunde situațiilor reale din educație, unde autoritatea este contextuală.

Middleware-urile verifică permisiunile înainte ca handler-ul să apeleze use case-ul sensibil. Totuși, verificarea nu se limitează la middleware. Use case-urile importante repetă sau specializează validarea atunci când decizia depinde de starea resursei. Această dublă atenție este justificată deoarece permisiunile sunt o zonă de risc major. Interfața poate ascunde un buton, middleware-ul poate bloca ruta, iar use case-ul poate refuza operația dacă resursa nu se află în starea corectă.

5.15 Implementarea recompenselor ca proces

Subdomeniul recompenselor este unul dintre cele mai elaborate deoarece combină pedagogie, economie și securitate. Implementarea pornește de la candidatul la recompensă. Candidatul conține sursa, evenimentul, actorii relevanți, starea și cheia de idempotentă. După creare, el poate fi evaluat de profesor, apoi de platformă. În această etapă se stabilește suma aprobată și politica aplicată. Fiecare decizie produce audit, astfel încât istoricul să poată fi reconstruit.

După aprobare, sistemul creează o intenție de plată sau un job de execuție. Această separare este importantă deoarece o plată tokenizată poate depinde de disponibilitatea contractului, de nonce, de gas, de semnătură și de confirmarea rețelei. Dacă etapa token reușește, portofelul intern este creditat. Dacă portofelul este creditat, utilizatorul primește notificare. Dacă una dintre etape eșuează, candidatul poate intra într-o stare de reconciliere, nu într-o stare ambiguă.

Tabela 5.1. Etapele implementate pentru recompensa LearnToken (elaborare proprie).

Etapă	Responsabilitate	Motiv arhitectural
Candidat	Leagă evenimentul educațional de recompensă	Separă învățarea de plata efectivă
Decizie profesor	Validează meritul educațional	Păstrează context pedagogic
Aprobare sumă	Aplică politică și control plat-formă	Previne recompense necontrolate
Execuție token	Confirmă operația cu LearnToken	Adaugă audit extern
Creditare wallet	Reflectă valoarea în aplicație	Permite dashboard și rapoarte
Reconciliere	Repară efecte parțiale	Evită dublarea și pierderea stării

5.16 Integrarea contractelor inteligente

Contractele sunt integrate prin două direcții: deploy și apeluri operaționale. La pornire sau în mediul local, aplicația poate inițializa adresele necesare pentru LearnToken și contractele auxiliare. Pentru operațiile de import sau transfer semnat, backend-ul pregătește datele, respectă chain id-ul, folosește ABI-ul generat și verifică rezultatele. Contractele rămân responsabile pentru reguli on-chain precum owner, nonce, permit și validarea semnăturii.

O decizie importantă este folosirea standardelor existente. ERC-20 oferă interoperabilitate, OpenZeppelin reduce riscul implementării greșite a funcțiilor de bază, iar EIP-712/EIP-2612 îmbunătățesc experiența semnăturilor [26, 22, 32]. Sistemul nu inventează un token complet nou, ci extinde un model cunoscut. Această alegere este defensivă și justificată academic: într-un domeniu cu risc financiar, reutilizarea standardelor mature este preferabilă originalității inutile.

5.17 Implementarea portofelelor

Portofelul intern este o oglindă operațională a valorii atribuite utilizatorului sau organizației în platformă. El nu înlocuiește portofelul blockchain, ci îl completează. Utilizatorul poate avea o adresă externă, dar aplicația are nevoie și de un ledger intern pentru istoricul recompenselor, taxelor, compensărilor, depunerilor și retragerilor. Acest ledger permite rapoarte, filtrare și reconciliere rapidă.

Implementarea portofelelor diferențiază actorii. Un wallet de utilizator are alt context decât un wallet de organizație. Organizațiile pot sponsoriza programe și pot avea audit financiar separat. De asemenea, platforma poate aplica taxe pentru anumite operații dacă suportă costul de gas. Această modelare arată că portofelul nu este doar o adresă Ethereum, ci o componentă a produsului educațional.

5.18 Worker-ul și toleranța la eșec

Worker-ul este construit pentru operații care nu trebuie să blocheze răspunsul HTTP. Procesarea video este exemplul principal: fișierul poate fi mare, transcodarea poate dura, iar rezultatul trebuie încărcat în S3. Dacă această operație ar rula în cererea profesorului, platforma ar fi fragilă. Prin joburi asincrone, profesorul primește o stare, iar sistemul poate procesa materialul în fundal.

Toleranța la eșec se bazează pe statusuri, retry-uri și heartbeat. Un job poate eșua temporar din cauza unui fișier lipsă, a unei probleme S3 sau a unei erori de transcodare. Sistemul trebuie să păstreze eroarea și să permită reluarea controlată. Heartbeat-ul arată că worker-ul este activ, iar readiness-ul general poate indica dacă platforma are toate dependențele necesare. Astfel, operarea nu depinde doar de loguri manuale.

5.19 Implementarea frontend-ului pe stări

Frontend-ul nu trebuie să presupună că toate datele sunt disponibile instantaneu. Fiecare ecran are stări de încărcare, eroare, conținut gol și conținut disponibil. Acest model este important pentru interfețele educaționale deoarece utilizatorul trebuie să înțeleagă de ce nu vede un curs, de ce nu poate aproba o recompensă sau de ce un wallet nu afișează încă tranzacția. O stare goală bine formulată poate preveni confuzia mai eficient decât o simplă listă vidă.

Interfața citește sesiunea curentă și capabilitățile asociate. Acțiunile sunt afișate în funcție

de permisiuni, nu doar de tipul paginii. De exemplu, un utilizator aflat în zona `/teach` poate avea acces la un curs, dar nu neapărat la toate operațiile de recompensare. Aceeași logică se aplică pentru organizații și admin. Astfel, frontend-ul devine o extensie a modelului de permisiuni, nu un strat independent care poate crea așteptări false.

5.20 KYC, audit și operații administrative

KYC-ul este inclus ca flux administrativ deoarece recompensele tokenizate pot cere verificări suplimentare în scenarii reale. Chiar dacă implementarea de disertație poate fi locală, arhitectura prevede statusuri, review, audit și decizii. Acest lucru permite extinderea ulterioară către furnizori reali fără a schimba toate fluxurile de portofel.

Auditul este prezent în mai multe domenii: aplicații de profesor, organizații, recompense, politici și portofele. El nu este doar o listă de loguri tehnice, ci o parte a modelului de produs. Când o decizie este contestată, platforma trebuie să poată răspunde cine a decis, când, cu ce motiv și ce stare avea resursa. Această capacitate este esențială pentru încrederea într-un sistem care combină educația cu tokenuri.

5.21 Observabilitate și administrare locală

Implementarea include endpoint-uri de health și readiness, comenzi `Make` și mediu `Docker Compose`. Aceste elemente creează o experiență reproductibilă pentru dezvoltare și verificare. `make test`, `make preflight`, `make dev`, `make dev-worker` și comenzile de migrare definesc pași clari. Într-un proiect aplicativ, această claritate este aproape la fel de importantă ca implementarea funcțiilor, deoarece permite evaluatorului sau dezvoltatorului să reproducă rezultatul.

Observabilitatea poate fi extinsă în viitor cu metrice, tracing și dashboard-uri. Totuși, chiar și forma actuală arată o direcție corectă: aplicația nu pornește ca un proces opac, ci își declară starea dependențelor. Worker-ul nu este presupus sănătos doar pentru că procesul există, ci scrie heartbeat. Aceste detalii reduc riscul ca platforma să pară funcțională în timp ce componentele critice sunt indisponibile.

5.22 Fluxuri concrete de API

Implementarea API-ului este gândită pentru a păstra rutele apropiate de limbajul produsului. Rutele de learning expun catalog, detaliu de curs, progres, evaluări, înscrieri și administrare de conținut. Rutele de teacher expun workspace-ul profesorului, cererile de înscriere, studenții și candidații la recompense. Rutele de rewards expun istoricul, politicile, candidații, deciziile și fraud blocks. Rutele de wallet expun legarea portofelului, auditul, depunerile, retragerile și taxele. Această separare face API-ul mai ușor de folosit și documentat.

Un avantaj al acestei organizări este că permisiunile pot fi aplicate aproape de resursa protejată. O rută de curs conține identificatorul cursului și poate verifica permisiuni de curs. O rută de organizație conține identificatorul organizației și poate verifica rolul organizațional. O rută de platformă cere drepturi globale. Dacă toate operațiile ar fi grupate generic sub un endpoint de administrare, această claritate s-ar pierde.

5.23 Validarea datelor la intrare

Datele primite prin API trebuie validate înainte de a ajunge în domeniu. Validarea include câmpuri obligatorii, lungimi, enum-uri, identificatori, sume și formate. De exemplu, o sumă de recompensă trebuie să fie pozitivă și compatibilă cu tipul numeric folosit în bază de date. O adresă de portofel trebuie să aibă format acceptabil. O decizie de profesor trebuie să conțină motiv atunci când respinge un candidat. Această validare previne erori mai jos în stivă.

Validarea nu înlocuiește regulile de domeniu. Chiar dacă un request este bine format, operația poate fi invalidă. Un curs poate fi arhivat, o politică poate fi inactivă, un fraud block poate opri recompensa sau actorul poate avea permisiune insuficientă. De aceea, RustLearn separă validarea formei de validarea sensului. Prima aparține stratului HTTP și DTO-urilor, a doua aparține use case-urilor și domeniului.

5.24 Implementarea exporturilor și rapoartelor

Exporturile CSV și rapoartele sunt utile deoarece datele platformei trebuie analizate în afara interfeței curente. Administratorii pot avea nevoie de situații privind recompensele în așteptare, wallet-urile organizațiilor, fraud blocks active sau progresul cursurilor sponsorizate. Implementarea exporturilor trebuie să respecte aceleași permisiuni ca interfața. Un export nu trebuie să devină

o scurtătură pentru acces la date nepermise.

Din punct de vedere tehnic, exportul transformă rezultate de use case în format tabular. Această transformare trebuie să fie explicită: coloane stabile, valori clare, date formate și lipsa secretelor. Pentru o platformă educațională, exportul este important și în procesul de evaluare, deoarece profesorii și organizațiile pot folosi datele pentru decizii. Astfel, raportarea este o funcție de produs, nu doar o utilitate administrativă.

5.25 Corelarea frontend-backend

Frontend-ul și backend-ul trebuie să evolueze împreună. Backend-ul definește contractele de date, iar frontend-ul le transformă în interfețe. Dacă backend-ul returnează stări de recompensă prea tehnice, frontend-ul trebuie să le traducă în mesaje utile. Dacă frontend-ul cere un câmp care nu are sens pentru un rol, contractul API trebuie ajustat sau explicat. Această corelare este vizibilă în tipurile din `web/src/lib`, unde rutele frontend folosesc funcții dedicate pentru sesiune, teacher, learner, organizații și wallet.

Un beneficiu al acestei structuri este izolarea erorilor. Dacă o rută de reward își schimbă răspunsul, adaptarea se face în biblioteca frontend aferentă, nu în toate componentele. Dacă sesiunea primește o capabilitate nouă, verificarea poate fi reutilizată în mai multe pagini. Această disciplină reduce duplicarea și susține interfețe coerente pentru roluri diferite.

5.26 Configurare și separarea mediilor

Configurarea aplicației este tratată ca parte a implementării, nu ca detaliu secundar. Cheile JWT, conexiunile PostgreSQL, endpoint-urile S3, adresele contractelor, cheia deployer-ului și setările worker-ului trebuie furnizate prin variabile de mediu sau secrete, nu prin cod sursă. Această separare permite rularea aceluiași cod în dezvoltare, testare, staging și producție, cu configurații diferite. În plus, reduce riscul ca secretele să fie comise accidental.

Separarea mediilor este importantă și pentru blockchain. În local se poate folosi Anvil, unde tokenurile nu au valoare reală și contractele pot fi redeployate ușor. În staging se poate folosi o rețea de test, cu date apropiate de producție, dar fără risc financiar major. În producție, contractele trebuie auditate, adresele trebuie controlate și secretele trebuie gestionate prin infrastructură specializată. Această trecere graduală este o condiție pentru o platformă care ar putea administra recompense cu valoare reală.

5.27 Calitatea codului și verificări înainte de livrare

Calitatea implementării este susținută prin format, teste și preflight. Formatul uniform reduce diferențele inutile în review. Testele verifică reguli de domeniu, API-uri, contracte și fluxuri de produs. Preflight-ul combină mai multe verificări pentru a oferi o imagine rapidă asupra stării repository-ului. În plus, migrațiile sunt rulate prin Make/Compose, ceea ce evită dependența de o instalare locală Diesel diferită de mediul proiectului.

Aceste verificări sunt relevante pentru disertație deoarece arată că sistemul nu este doar o descriere teoretică. El are o disciplină de dezvoltare care permite modificări repetate fără pierderea controlului. Într-un proiect cu recompense, permisiuni și contracte, această disciplină este esențială. O eroare de format este minoră, dar lipsa unei culturi de verificare poate ascunde erori de autorizare, migrații incomplete sau stări financiare incorecte.

5.28 Concluzie

Implementarea RustLearn urmărește separarea clară a fluxurilor. Backend-ul expune API-uri protejate, use case-urile gestionează regulile, domeniul păstrează concepte stabile, infrastructura conectează PostgreSQL, S3 și Ethereum, iar frontend-ul oferă interfețe adaptate rolurilor. Contractele inteligente adaugă un strat verificabil pentru LearnToken, iar worker-ul asigură operații asincrone. Această implementare susține obiectivul lucrării: un sistem de eLearning în care recompensele blockchain sunt integrate controlat și auditat.

Capitolul 6

Validare, operare și limitări

6.1 Strategia de validare

Validarea platformei urmărește două direcții: validarea tehnică și validarea fluxurilor de produs. Validarea tehnică verifică dacă aplicația compilează, dacă migrațiile sunt coerente, dacă testele unitare și de integrare rulează, dacă frontend-ul trece testele și dacă serviciile locale pornesc în Docker Compose. Validarea fluxurilor de produs verifică scenarii precum autentificarea, accesul la cursuri, aprobarea profesorilor, administrarea recompenselor, creditarea portofelelor, raportarea și operarea worker-ului.

Repository-ul include comenzi Make pentru format, teste, preflight, testare în Compose, migrații și pornirea mediului de dezvoltare. Această alegere este importantă deoarece reduce dependența de pași manuali. În loc ca fiecare dezvoltator să ghicească ce comandă trebuie rulată, Makefile-ul devine interfața operațională a proiectului. Pentru o lucrare aplicativă, această disciplină arată că sistemul nu este doar cod demonstrativ, ci un produs care poate fi rulat și verificat.

6.2 Teste backend

Backend-ul are teste pentru domenii critice, inclusiv recompense, permisiuni, tokenuri, integrarea blockchain și adaptoare. Testele unitare verifică reguli izolate, cum ar fi parsarea tipurilor de evenimente token sau validarea unor stări de recompensare. Testele de integrare verifică fluxuri mai largi, unde baza de date, API-ul sau Ethereum local pot interacționa. Pentru contracte, testele Foundry verifică funcțiile LearnToken, LearnTokenPresigner și PlatformImporter.

Tabelul 6.1 sintetizează principalele niveluri de validare.

Tabela 6.1. Niveluri de validare pentru RustLearn (elaborare proprie).

Nivel	Ce verifică	Exemple de risc redus
Unit	Reguli de domeniu, validări, conversii de stare	Stări imposibile, parse greșit, calcule incorecte
Integrare API	Rute, middleware, permisiuni, persistență	Acces neautorizat, erori de contract API, date lipsă
Blockchain	Contracte, evenimente, semnături, deploy local	Replay, emitere necontrolată, ABI incompatibil
Frontend	Randare rute, stări goale, controale gated	Interfețe care afișează acțiuni fără permisiuni
Operațional	Health, readiness, Docker Compose, worker	Servicii pornite fără dependențe reale sau blocaje ascunse

6.3 Migrații și schema bazei de date

Migrațiile Diesel definesc evoluția bazei de date. Fiecare schimbare de schemă trebuie să aibă `up.sql` și `down.sql`, astfel încât evoluția să fie reversibilă când este posibil. Pentru o platformă cu recompense și portofele, migrațiile nu sunt doar detalii tehnice. Ele definesc structura de audit, tranzacții, stări și constrângeri de unicitate. De exemplu, cheile de idempotentă și indexurile unice reduc riscul dublării unor operații.

Schema curentă include tabele pentru candidați la recompense, politici, joburi de execuție, payout records, wallet credit records, compensation records, fraud blocks și audit events. Această separare permite interogări și raportări precise. Dacă toate aceste informații ar fi păstrate într-un singur câmp JSON, dezvoltarea inițială ar fi mai rapidă, dar verificarea și reconcilierea ar deveni mai dificile.

6.4 Health și readiness

RustLearn expune `/health` pentru liveness și `/ready` pentru dependențe. Diferența este importantă. Un proces poate fi viu, dar nepregătit să servească trafic dacă baza de date, S3 sau Ethereum nu sunt disponibile. Readiness-ul verifică dependențele înainte ca platforma să fie considerată

operațională. Frontend-ul poate afișa aceste stări în dashboard-ul de administrare, iar Kubernetes sau Docker pot folosi endpoint-urile pentru health checks.

Worker-ul are propriul mecanism de heartbeat. El scrie periodic un fișier care poate fi verificat de health check. Dacă worker-ul se blochează, lipsa heartbeat-ului indică problema. Această verificare este utilă deoarece worker-ul nu expune neapărat un server HTTP, dar este critic pentru procesarea video și indexarea depunerilor.

6.5 Docker Compose și mediu local

Mediul local include serviciile `app`, `web`, `db`, `rustfs`, `anvil`, `worker` și `test-runner`. PostgreSQL rulează ca bază de date, RustFS oferă stocare compatibilă S3, Anvil simulează rețeaua Ethereum, iar worker-ul rulează sarcinile asincrone. Această compunere permite testarea locală a fluxurilor care, în producție, ar depinde de servicii externe.

Valoarea Docker Compose este reproducibilitatea. O platformă care integrează DB, S3, blockchain și worker poate deveni greu de pornit manual. Compose definește dependențele, variabilele și porturile într-un singur loc. În plus, același model conceptual poate fi transpus în Kubernetes, unde readiness-ul, secretele, volumele și restart policy devin elemente de operare.

6.6 Securitate și control al riscurilor

Riscurile principale sunt accesul neautorizat, dublarea recompenselor, manipularea portofelelor, expunerea secretelor și inconsistența între baza de date și blockchain. Sistemul reduce aceste riscuri prin JWT, permisiuni pe scopuri, audit, stări de recompensare, politici versionate, nonce-uri în contracte, deadline-uri pentru semnături, chei de idempotentă și reconciliere.

Un risc specific aplicațiilor blockchain este tentația de a considera blockchain-ul suficient pentru audit. În realitate, blockchain-ul poate confirma o tranzacție, dar nu explică toate deciziile educaționale care au dus la ea. De aceea, RustLearn păstrează auditul deciziilor în PostgreSQL și leagă tranzacțiile externe de candidați, portofele și politici. Această legătură este necesară pentru investigații și rapoarte.

6.7 Limitări ale implementării

Implementarea curentă are limitări. În primul rând, unele fluxuri frontend marchează explicit funcții viitoare, cum ar fi progresul persistent complet, încărcarea media avansată sau anumite detalii de buget. În al doilea rând, emailul este încă simulat local, nu integrat cu un furnizor real. În al treilea rând, contractele nu sunt upgradeable proxies, ceea ce înseamnă că schimbările majore de ABI necesită redeploy și plan de reconciliere. În al patrulea rând, utilizarea tokenurilor în producție ar necesita audit de securitate dedicat și politici juridice clare.

O altă limitare este lipsa unei evaluări cu utilizatori reali. Lucrarea validează arhitectura și implementarea tehnică, dar nu măsoară încă impactul recompenselor asupra motivației sau rezultatelor educaționale. Pentru o cercetare completă de produs, ar fi necesar un studiu cu grupuri de cursanți, comparație între cursuri cu și fără recompense și măsurarea retenției, promovării și satisfacției.

6.8 Scenarii de testare recomandate

Pentru verificare completă, sunt recomandate următoarele scenarii:

- înregistrarea unui cursant, verificarea emailului, autentificarea și accesarea dashboard-ului `/learn`;
- aplicarea pentru rol de profesor, aprobarea de către platformă și accesarea rutelor `/teach`;
- crearea unui curs, adăugarea de capitole și conținut, publicarea și solicitarea de înscriere de către cursant;
- completarea unei evaluări, generarea candidatului la recompensă și aprobarea de către profesor;
- aprobarea sumei la nivel de platformă, planificarea plății și creditarea portofelului;
- încărcarea unui video și verificarea procesării prin worker;
- simularea unei stări inconsistente și rularea reconcilierii;
- verificarea interfețelor fără permisiuni, pentru a confirma că acțiunile sensibile nu sunt afișate sau executate.

6.9 Criterii de acceptanță

Pentru ca platforma să fie considerată validată la nivel de disertație, nu este suficient ca serverul să pornească. Trebuie definite criterii de acceptanță pentru fluxurile centrale. Un cursant trebuie să se poată autentifica, să vadă catalogul, să acceseze un curs permis, să parcurgă conținutul și să primească feedback pentru evaluări. Un profesor trebuie să poată crea cursuri, să gestioneze conținut, să observe studenți și să decidă candidați la recompense. Un administrator trebuie să poată gestiona politici, fraud blocks și rapoarte fără a accesa manual baza de date.

La nivel tehnic, criteriile includ rularea testelor relevante, aplicarea migrațiilor, pornirea mediului local, verificarea readiness-ului și generarea artefactelor blockchain. La nivel de produs, criteriile includ mesaje clare de eroare, stări goale inteligibile și restricționarea acțiunilor în funcție de permisiuni. Această dublă perspectivă este necesară deoarece o platformă poate fi corectă tehnic, dar dificil de folosit, sau poate avea o interfață plăcută, dar reguli nesigure.

Tabela 6.2. Criterii de acceptanță pentru principalele fluxuri (elaborare proprie).

Flux	Criteriu funcțional	Criteriu de siguranță
Autentificare	Utilizatorul primește sesiune validă și poate verifica profilul	Tokenurile expiră, parola respectă politica
Cursuri	Cursurile publicate sunt vizibile în catalog	Draft-urile rămân ascunse fără permisiune
Evaluări	Scorul este calculat și progresul se actualizează	Evenimentele nereușite nu creează recompense
Recompense	Candidatul parcurge aprobări și creditare	Idempotenta previne dublarea recompensei
Wallet	Istoricul arată tranzacțiile relevante	Operațiile externe sunt legate de audit intern
Worker	Video-ul este procesat sau eșecul este raportat	Joburile pot fi reluate controlat

6.10 Matrice de riscuri

O platformă care combină educația și tokenizarea are riscuri tehnice, operaționale și etice. Riscurile tehnice includ buguri în contracte, pierderea sincronizării cu blockchain-ul, migrații greșite sau erori de permisiuni. Riscurile operaționale includ indisponibilitatea S3, creșterea cozii worker-ului, costuri de gas mai mari decât estimarea și lipsa monitorizării. Riscurile etice includ stimulente greșite, recompense disproporționate și confuzie între valoarea educațională și valoarea economică.

Tabela 6.3. Matrice sintetică de riscuri și controale (elaborare proprie).

Risc	Impact	Control propus
Dublare recompensă	Pierdere financiară și neîncredere	Idempotentă, stări, audit și reconciliere
Permisiiune excesivă	Acces la date sau operații neautorizate	Scopuri de permisiuni și teste dedicate
Contract vulnerabil	Pierdere tokenuri sau blocare fonduri	OpenZeppelin, teste Foundry, audit extern
Worker blocat	Video-uri neprocesate și cozi în creștere	Heartbeat, retry și dashboard operațional
Date personale expuse	Risc legal și reputațional	Păstrare off-chain, minimizare și acces controlat
Motivație distorsionată	Învățare superficială	Recompense după validare educațională

6.11 Testare manuală end-to-end

Testarea manuală end-to-end completează testele automate deoarece verifică experiența între ecrane și servicii. Un scenariu complet pornește cu un utilizator nou, continuă cu verificarea emailului, înscrierea la un curs, parcurgerea unei lecții, susținerea evaluării, generarea candidatului la recompensă, decizia profesorului, aprobarea sumei și verificarea wallet-ului. În acest traseu pot fi observate mesaje, încărcările, stările goale și restricțiile vizuale.

Pentru profesor, testarea manuală trebuie să includă crearea unui curs, încărcarea unui fișier, publicarea conținutului, vizualizarea studenților și decizia asupra candidaților. Pentru administra-

tor, scenariile includ crearea unei politici, activarea sau dezactivarea ei, aplicarea unui fraud block, consultarea auditului și exportul datelor. Aceste scenarii pot fi documentate ca script de verificare înainte de prezentarea lucrării.

6.12 Plan de deployment

Deployment-ul unei platforme RustLearn presupune cel puțin următoarele componente: API, frontend, PostgreSQL, stocare S3 compatibilă, Ethereum node sau provider, worker și mecanisme de secrete. În dezvoltare, Docker Compose oferă un mediu suficient. În producție, Kubernetes poate gestiona rollout-uri, readiness probes, volume, secrete și restart policy. Totuși, trecerea la producție trebuie făcută gradual, cu mediu de staging și rețea de test pentru contracte.

Un plan rezonabil include trei etape. Prima etapă este mediul local reproductibil, folosit pentru dezvoltare și testare. A doua etapă este staging, cu servicii similare producției, dar fără valoare reală în tokenuri. A treia etapă este producția, unde secretele sunt administrate separat, contractele sunt auditate, backup-urile sunt testate, iar monitorizarea este activă. Această etapizare reduce riscul ca o eroare de dezvoltare să devină incident financiar.

6.13 Backup, restaurare și migrații

PostgreSQL este sursa principală pentru starea educațională și auditul operațional. Backup-ul bazei de date trebuie tratat ca funcție critică. O strategie minimă include backup-uri automate, verificarea restaurării și păstrarea unor copii în locații separate. Pentru S3, obiectele media trebuie versionate sau cel puțin protejate prin politici de retenție, deoarece pierderea conținutului poate afecta cursuri publicate. Pentru contracte, adresele și ABI-urile trebuie păstrate împreună cu versiunea aplicației.

Migrațiile trebuie rulate controlat. Înainte de o migrare majoră, se verifică schema în staging și se rulează testele relevante. Dacă migrarea introduce tabele pentru recompense sau portofele, trebuie analizat impactul asupra datelor existente. Un `down.sql` reversibil este util, dar nu înlocuiește backup-ul. În practică, restaurarea datelor este ultima linie de apărare, iar migrațiile trebuie proiectate astfel încât să reducă nevoia de rollback.

6.14 Performanță și scalare

Performanța RustLearn depinde de trei zone: API-ul, baza de date și operațiile asincrone. API-ul Rust/Actix Web poate susține concurență bună, dar performanța reală este limitată de interogări, indexuri și servicii externe. PostgreSQL trebuie optimizat pentru interogări frecvente: catalogul cursurilor, progresul cursantului, coada de candidați și istoricul wallet-ului. Worker-ul trebuie dimensionat în funcție de numărul și mărimea fișierelor video.

Scalarea trebuie făcută pe baza măsurătorilor. Dacă API-ul este bottleneck, se pot adăuga replici. Dacă baza de date este bottleneck, se analizează indexuri, pool-uri și interogări. Dacă procesarea video este bottleneck, se mărește numărul de worker-e sau se separă tipurile de joburi. Dacă blockchain-ul este bottleneck, platforma poate grupa operații, poate folosi rețele potrivite sau poate muta unele acțiuni în fluxuri semnate. Important este că arhitectura are puncte clare de scalare.

6.15 Conformitate și protecția datelor

Datele educaționale pot fi sensibile. Ele includ progres, rezultate la evaluări, relații cu organizații, cereri de profesor și eventual informații KYC. De aceea, decizia de a nu scrie aceste date pe blockchain este importantă. Blockchain-ul este dificil de modificat sau șters, iar transparența lui poate intra în conflict cu protecția datelor personale. RustLearn păstrează aceste informații în PostgreSQL, unde accesul poate fi controlat, auditat și limitat.

Conformitatea viitoare ar presupune politici de retenție, export de date personale, ștergere sau anonimizare acolo unde legea o cere, consimțământ pentru prelucrare și documentarea scopurilor. Pentru tokenuri, pot apărea obligații suplimentare juridice sau fiscale, în funcție de valoarea reală și de modul de utilizare. Lucrarea nu rezolvă complet aceste aspecte, dar arhitectura le lasă spațiu: datele sensibile sunt off-chain, auditul este intern, iar fluxurile pot fi extinse cu aprobări și controale.

6.16 Limitări de cercetare și validare empirică

Validarea tehnică arată că platforma poate fi construită și operată, dar nu dovedește singură eficiența pedagogică a recompenselor. Pentru a evalua impactul real, ar fi necesar un studiu cu utilizatori. Un design posibil ar compara două grupe care parcurg același curs: una cu recompense

tokenizate și una fără. Indicatorii ar putea include rata de finalizare, timpul petrecut în platformă, numărul de reveniri, scorul la evaluări și satisfacția raportată.

Un astfel de studiu trebuie proiectat atent pentru a evita concluzii superficiale. Dacă grupa cu recompense finalizează mai multe lecții, nu înseamnă automat că a învățat mai bine. Trebuie analizate rezultatele evaluărilor și calitatea participării. De asemenea, recompensele trebuie calibrate astfel încât să nu transforme cursul într-un simplu mecanism de câștig. Această limitare este recunoscută în lucrare și constituie una dintre direcțiile viitoare importante.

6.17 Indicatori recomandați pentru monitorizare

Într-un mediu real, monitorizarea trebuie să acopere atât partea tehnică, cât și partea de produs. Indicatorii tehnici includ timpul de răspuns al API-ului, erorile HTTP, conexiunile PostgreSQL, latența S3, mărimea cozii worker-ului, numărul de joburi eșuate și întârzierea indexerului blockchain. Indicatorii de produs includ rata de finalizare a cursurilor, numărul de candidați la recompensă, rata de respingere, timpul mediu până la creditarea wallet-ului și numărul de fraud blocks active.

Acești indicatori trebuie interpretați împreună. O creștere a candidaților la recompensă poate indica succes educațional, dar poate indica și abuz dacă este însoțită de multe fraud blocks. O scădere a finalizărilor poate indica dificultate reală a cursului sau probleme de interfață. O creștere a joburilor video eșuate poate indica fișiere invalide sau limitări de infrastructură. Monitorizarea valoroasă nu se limitează la alarme, ci oferă context pentru decizii.

6.18 Pregătirea susținerii și demonstrației

Pentru susținerea disertației, demonstrația trebuie să evidențieze mai ales partea a II-a, conform ghidului. O demonstrație potrivită poate porni de la autentificare, apoi poate prezenta catalogul, cursul, rolul profesorului, candidatul la recompensă, aprobarea și wallet-ul. Este important ca demonstrația să nu fie doar o navigare prin ecrane, ci o explicație a deciziilor arhitecturale: de ce recompensa are stări, de ce profesorul nu plătește direct, de ce tokenul este separat de datele educaționale și de ce readiness-ul verifică dependențe.

În prezentare, accentul trebuie pus pe contribuții: arhitectura hibridă, modelul pe scopuri pentru permisiuni, recompensele auditate, integrarea contractelor și operabilitatea. Capturile de ecran, diagramele și fragmentele de cod din lucrare pot fi folosite ca suport. O prezentare bună tre-

buie să lege fiecare funcție demonstrată de problema inițială: cum poate fi recompensată activitatea educațională fără a pierde controlul, securitatea și sensul pedagogic.

6.19 Criterii de maturitate pentru continuarea proiectului

După etapa de disertație, proiectul ar trebui evaluat printr-un set de criterii de maturitate. Primul criteriu este acoperirea fluxurilor centrale prin teste automate și scenarii manuale repetabile. Al doilea criteriu este stabilitatea datelor: migrațiile trebuie să fie clare, backup-ul testat, iar exporturile să poată explica deciziile importante. Al treilea criteriu este securitatea: contractele trebuie auditate, permisiunile revizuite, iar secretele mutate în infrastructură dedicată. Al patrulea criteriu este validarea pedagogică prin utilizatori reali.

Aceste criterii arată drumul de la lucrare aplicativă la produs. RustLearn are baza arhitecturală: domenii separate, audit, worker, readiness, contracte și frontend pe roluri. Pentru producție, această bază trebuie completată cu monitorizare, politici juridice, suport, proceduri de incident și analiză de impact educațional. Astfel, concluzia validării nu este că sistemul este terminat definitiv, ci că are o fundație suficient de solidă pentru dezvoltare controlată.

6.20 Concluzie

Validarea RustLearn trebuie privită ca un proces continuu. Sistemul are mecanisme tehnice solide: teste, migrații, health checks, readiness checks, worker heartbeat și mediu local compus. Totuși, integrarea blockchain introduce riscuri suplimentare, iar trecerea în producție ar necesita audit de securitate, monitorizare, backup-uri și politici clare de operare. Arhitectura existentă creează baza pentru aceste etape, deoarece stările, auditul și separarea responsabilităților sunt deja prezente.

Capitolul 7

Concluzii și direcții viitoare

7.1 Sinteza lucrării

Lucrarea a avut ca obiectiv proiectarea și implementarea unui sistem de eLearning bazat pe blockchain, în limba română, conform cerințelor academice pentru o disertație. Tema a fost abordată prin analiza domeniului eLearning, a gamificării, a modelului learn-to-earn, a riscurilor de tokenizare educațională și a aplicațiilor hibride Web2/Web3, apoi prin proiectarea și documentarea platformei RustLearn. Rezultatul este un sistem care combină un backend Rust/Actix Web, o bază PostgreSQL, stocare S3, worker asincron, contracte Solidity și frontend Next.js.

Concluzia principală este că blockchain-ul aduce valoare într-o platformă educațională atunci când este folosit selectiv. Nu toate datele educaționale trebuie puse on-chain. Lecțiile, progresul, permisiunile, evaluările, rapoartele și auditul operațional sunt mai potrivite într-o infrastructură controlată, rapidă și relațională. În schimb, tokenurile, transferurile și evenimentele financiare verificabile pot beneficia de transparența și interoperabilitatea Ethereum. Această separare este esențială pentru caracterul practic al soluției.

7.2 Contribuții realizate

Prima contribuție este definirea unei arhitecturi hibride pentru eLearning recompensat. Arhitectura arată cum pot coexista un sistem centralizat de cursuri și un strat blockchain de recompense fără ca unul să îl compromită pe celălalt. Backend-ul rămâne sursa de adevăr pentru reguli, iar blockchain-ul devine strat de verificare și transfer pentru tokenuri.

A doua contribuție este modelarea recompensei ca proces auditat. În loc să se acorde automat tokenuri la primul eveniment, RustLearn folosește candidați, politici, aprobări, planificare

de payout, confirmare token, creditare wallet, notificare și reconciliere. Acest model este mai potrivit pentru o platformă reală deoarece acceptă erori, întârzieri și decizii administrative fără a pierde trasabilitatea.

A treia contribuție este separarea permisiunilor pe scopuri: platformă, organizație și curs. Această separare reflectă mai bine realitatea educațională decât un model global simplu. Un utilizator poate avea rol de profesor într-un curs, rol de membru într-o organizație și rol de cursant în alt context. Arhitectura permite exprimarea acestor situații fără privilegii excesive.

A patra contribuție este integrarea operațională a componentelor. Sistemul include migrații, health checks, readiness checks, Docker Compose, worker heartbeat, teste și documentație de operare. Aceste elemente fac diferența între un prototip izolat și o platformă care poate fi rulată, verificată și extinsă.

7.3 Răspuns la problema inițială

Problema inițială a fost construirea unui sistem de eLearning bazat pe blockchain care să fie util, sigur și justificat arhitectural. Soluția propusă răspunde acestei probleme printr-o arhitectură modulară monolitică și hibridă. Monolitul modular reduce complexitatea operațională, iar separarea internă păstrează domeniile clare. Integrarea blockchain este limitată la tokenuri și fluxuri de valoare, ceea ce evită costuri inutile și protejează datele educaționale.

Tehnologiile alese sunt coerente cu această problemă. Rust oferă siguranță și performanță, Actix Web oferă un API modular, PostgreSQL și Diesel oferă consistență relațională, S3 susține conținut multimedia, worker-ul separă operațiile costisitoare, Next.js oferă suprafețe de produs clare, iar Solidity/OpenZeppelin oferă contracte tokenizate mature. Fiecare tehnologie are un rol justificat în ansamblu.

7.4 Limitări

Lucrarea are limite care trebuie recunoscute. În primul rând, nu include un studiu cu utilizatori reali pentru a măsura impactul recompenselor asupra motivației. În al doilea rând, unele fluxuri sunt încă pregătite pentru dezvoltare viitoare, în special emailul real, media upload avansat și unele detalii de progres persistent. În al treilea rând, contractele inteligente ar necesita audit extern înainte de utilizare în producție cu valoare reală. În al patrulea rând, orice sistem learn-to-earn trebuie analizat și din perspectivă juridică, fiscală și etică.

Aceste limitări nu invalidează soluția, ci indică diferența dintre o platformă aplicativă de disertație și un produs complet lansat public. Arhitectura a fost concepută tocmai pentru a permite aceste extinderi fără schimbări fundamentale.

7.5 Direcții viitoare

O direcție viitoare este introducerea unui serviciu real de email, cu livrare, template-uri, retry și monitorizare. O altă direcție este extinderea progresului persistent, astfel încât fiecare lecție, activitate și evaluare să contribuie la un model mai bogat de învățare. De asemenea, platforma poate adăuga analytics educațional pentru profesori și organizații, cu indicatori privind retenția, dificultatea lecțiilor și rata de promovare.

Pe partea blockchain, direcțiile viitoare includ audit extern pentru contracte, suport pentru rețele de test publice, mecanisme de treasury mai avansate, politici de emitere mai stricte și interfețe mai clare pentru MetaMask sau alte portofele. Reconcilierea poate fi extinsă cu joburi programate, alerte și dashboard-uri pentru operatori. În plus, token burns și taxele de depunere sau retragere pot fi integrate într-un model economic mai formal.

Pe partea de produs, este necesar un studiu cu utilizatori reali. Platforma poate fi testată pe un set de cursuri pilot, comparând grupe cu recompense tokenizate și grupe fără recompense. Indicatorii relevanți ar putea include rata de finalizare, numărul de reveniri, rezultatele evaluărilor și percepția cursanților asupra utilității recompenselor. Acest studiu ar transforma lucrarea dintr-o contribuție tehnică într-o cercetare aplicată asupra impactului motivațional.

7.6 Concluzie finală

Sistemul de eLearning bazat pe blockchain prezentat în lucrare demonstrează că tokenizarea poate fi integrată într-o platformă educațională fără a sacrifica cerințele clasice de securitate, performanță și control. Valoarea soluției nu stă în folosirea blockchain-ului ca scop în sine, ci în folosirea lui acolo unde aduce transparentă și verificabilitate. Prin arhitectura hibridă, RustLearn oferă o bază solidă pentru cursuri, evaluări, organizații, portofele și recompense LearnToken, păstrând posibilitatea de extindere către scenarii reale de învățare recompensată și cercetare aplicată asupra motivației cursanților.

Bibliografie

- [1] V. Bell, C. Silva, J. A. Guina și T. Fernandes, „Community Health of Children and Adolescents in Sub-Saharan Africa”, în *European Journal of Medical and Health Sciences* 5 (Iun. 2023), pp. 22–31, DOI: 10.24018/ejmed.2023.5.3.1702.
- [2] E. Brown, *Web Development with Node and Express: Leveraging the JavaScript Stack*, O'Reilly Media, 2019.
- [3] G. Coulouris, J. Dollimore și T. Kindberg, *Distributed Systems: Concepts and Design*, 3rd, Pearson, Addison Wesley, 2001.
- [4] V. P. Dennen, B. M. Hall și A. Hedquist, „Instructional Strategies for Engaging Online Learners: Do Learner-centeredness and Modality Matter?”, în *Online Learning Journal* 27.1 (2022), pp. 158–186, DOI: 10.24059/olj.v27i1.3430.
- [5] S. Deterding, M. Sicart, L. Nacke, K. O'Hara și D. Dixon, „Gamification: Using Game Design Elements in Non-Gaming Contexts”, în *Proceedings of the 2011 Annual Conference on Human Factors in Computing Systems*, 2011, pp. 2425–2428, DOI: 10.1145/1979742.1979575, URL: <https://dl.acm.org/doi/10.1145/1979742.1979575>.
- [6] L. Freina și M. Ott, „A literature review on immersive virtual reality in education: State of the art and perspectives”, în *The International Scientific Conference eLearning and Software for Education*, vol. 1, ”Carol I” National Defence University, 2015, pp. 133–141.
- [7] S. Hrastinski, „A theory of online learning as online participation”, în *Computers & education* 52.1 (2009), pp. 78–82.
- [8] S. Klabnik și C. Nichols, *The Rust Programming Language*, No Starch Press, 2019, URL: <https://doc.rust-lang.org/book/>.
- [9] M. Liu, Y. Li și S. Li, „Web3: Exploring Decentralized Technologies and Applications for the Future of Empowerment and Ownership”, în *Blockchains* 1.2 (2023), pp. 111–131, DOI: 10.3390/blockchains1020008.

- [10] F. Martin și D. U. Bolliger, „Engagement matters: Student perceptions on the importance of engagement strategies in the online learning environment”, în *Online Learning Journal* 22.1 (2018), pp. 205–222, DOI: 10.24059/olj.v22i1.1092, URL: <https://doi.org/10.24059/olj.v22i1.1092>.
- [11] A. Miola, *Flutter Complete Reference: Create beautiful, fast and native apps for any device*, Packt Publishing, 2020.
- [12] C. Paans, I. Molenaar, E. Segers și L. Verhoeven, „Temporal variation in children’s self-regulated hypermedia learning”, în *Computers in Human Behavior* 84 (2018), pp. 307–319, DOI: 10.1016/j.chb.2018.04.002, URL: <https://doi.org/10.1016/j.chb.2018.04.002>.
- [13] J. Radianti, T. A. Majchrzak, J. Fromm și I. Wohlgenannt, „A systematic review of immersive virtual reality applications for higher education: Design elements, lessons learned, and research agenda”, în *Computers & Education* 147 (2020), p. 103778.
- [14] A. Sajedi, M. Mahdavi, A. Pour Shir Mohammadi și M. Monajjemi Nejad, „Fundamental Usability Guidelines for User Interface Design”, în *2008 International Conference on Computational Sciences and Its Applications*, 2008, pp. 106–113, DOI: 10.1109/ICCSA.2008.45, URL: <https://doi.org/10.1109/ICCSA.2008.45>.
- [15] K. Squire, *Video games and learning: Teaching and participatory culture in the digital age*, Teachers College Press, 2011.

Webografie

- [16] Amazon Web Services, *Amazon Simple Storage Service Documentation*, 2026, URL: <https://docs.aws.amazon.com/s3/> (accesat în 17.6.2026).
- [17] BitDegree, *BitDegree: Online Learning Platform*, 2024, URL: <https://www.bitdegree.org> (accesat în 17.6.2026).
- [18] Brave, *Introducing the Brave Wallet, a Browser-Native Crypto Wallet With No Extension Required*, 2021, URL: <https://brave.com/blog/brave-wallet-launch/> (accesat în 17.6.2026).
- [19] V. Buterin, *Ethereum Whitepaper*, 2014, URL: <https://ethereum.org/whitepaper/> (accesat în 17.6.2026).

- [20] CoinCodex, *Brave vs Chrome: Should You Switch to Crypto-Friendly Browser?*, 2024, URL: <https://coincodex.com/article/36577/brave-vs-chrome/> (accesat în 17.6.2026).
- [21] Diesel Project, *Diesel: Getting Started*, 2026, URL: <https://diesel.rs/guides/getting-started/> (accesat în 17.6.2026).
- [22] N. Johnson, F. Araoz, M. Evans, M. Lundkvist și A. Tomaino, *EIP-712: Typed Structured Data Hashing and Signing*, 2017, URL: <https://eips.ethereum.org/EIPS/eip-712> (accesat în 17.6.2026).
- [23] S. Klabnik, C. Nichols, C. Krycho și T. R. Community, *The Rust Programming Language*, Rust Project, 2026, URL: <https://doc.rust-lang.org/book/> (accesat în 17.6.2026).
- [24] Moralis, *Decentralized Applications Explained: What Are dApps?*, 2024, URL: <https://moralis.io/decentralized-applications-explained-what-are-dapps/> (accesat în 17.6.2026).
- [25] D. Novac, *The Basics of UI Design: Creating Intuitive User Interfaces*, UX Media, 2023, URL: <https://uxmedia.io/blog/what-is-user-interface-ui-design/> (accesat în 17.6.2026).
- [26] OpenZeppelin, *ERC-20 Documentation*, 2026, URL: <https://docs.openzeppelin.com/contracts/5.x/erc20> (accesat în 17.6.2026).
- [27] P. Pandya, *The Fundamentals of UI and UX Design: A Comprehensive Guide*, The Cuneiform, 2024, URL: <https://www.thecuneiform.com/insights/the-fundamentals-of-ui-and-ux-design-a-comprehensive-guide/> (accesat în 17.6.2026).
- [28] Phemex, *Learn Crypto: Educational Resources and Rewards*, 2024, URL: <https://phemex.com/learn-crypto> (accesat în 17.6.2026).
- [29] PostgreSQL Global Development Group, *PostgreSQL Documentation*, 2026, URL: <https://www.postgresql.org/docs/> (accesat în 17.6.2026).
- [30] Solidity Project, *Solidity Documentation*, 2026, URL: <https://docs.soliditylang.org/> (accesat în 17.6.2026).
- [31] The Actix Team, *Actix Web Documentation*, 2026, URL: <https://actix.rs/docs/> (accesat în 17.6.2026).

- [32] Uniswap și V. Buterin, *ERC-2612: Permit Extension for EIP-20 Signed Approvals*, 2020, URL: <https://eips.ethereum.org/EIPS/eip-2612> (accesat în 17.6.2026).
- [33] Vercel, *Next.js Docs: App Router*, 2026, URL: <https://nextjs.org/docs/app> (accesat în 17.6.2026).